

Node.js

130 Interview Questions & Answers

Event Loop — All 6 Phases & Order

Non-blocking I/O Deep Dive

CommonJS vs ES Modules

Streams, Buffers & Backpressure

Cluster & Worker Threads

process.nextTick vs setImmediate

EventEmitter & Event-Driven Design

Error Handling Best Practices

Graceful Shutdown

Rate Limiting & Security

Redis Caching Strategies

Connection Pooling

WebSockets & Real-time

PM2 in Production

AsyncLocalStorage

gRPC & GraphQL

Microservices Patterns

Lambda/Serverless

Performance Optimization

Memory Leak Detection

Child Processes & IPC

Cluster IPC

Logging with Pino

Docker & Kubernetes

HTTP/2 & TLS

→ All 57 Topic with production code examples ←

→ Follow for Next: Express.js (80 Q) → MongoDB (120 Q) → Auth & Security (60 Q) ←

■ Save • ■ Comment 'JS950' for full PDF • ■■ Repost

Kaushal Singh

Full-Stack Dev | MERN + React Native | Built Fintech Apps

1M+ Impressions | Follow for daily Node.js · Express · Interview Prep

■ Section 3/10 | 130 Q&A | Next: Express.js 80 Q | Comment 'JS950'

What is Node.js and how does it work internally?

■ Answer

Node.js is a JavaScript runtime built on Chrome's V8 engine that allows JS to run on the server. Created by Ryan Dahl in 2009 — revolutionized backend development with JS.

Internal architecture:

- V8 Engine — compiles JS to machine code (Google Chrome's engine)
- libuv — C library that provides event loop + async I/O + thread pool
- Node.js bindings — connect JS to C/C++ APIs
- Event Loop — handles async callbacks without blocking

Key properties:

- Single-threaded — one thread runs your JS code
- Non-blocking I/O — I/O doesn't block the thread
- Event-driven — program flow driven by events
- Fast — V8 compiles JS to native machine code

How a request is handled:

- Request arrives → pushed to event queue
- Event loop picks it up → starts processing
- I/O needed → handed to libuv (thread pool or OS)
- JS thread free to handle next request
- When I/O done → callback queued → event loop executes it

```
1 // Node.js internals visualization:
2 //
3 //   JavaScript Code (your app)
4 //       ↓
5 //   V8 Engine (compiles JS → machine code)
6 //       ↓
7 //   Node.js Bindings (C++ bridge)
8 //       ↓
9 //   libuv Library
10 //   ■■■ Event Loop (single thread)
11 //   ■■■ Thread Pool (4 threads, for heavy I/O)
12 //       ↓
13 //   Operating System (network, files)
14
15 // Simple server demonstrating event-driven model
16 const http = require('http')
17
18 const server = http.createServer((req, res) => {
19   // This callback fires when event loop processes the request
20   res.end(JSON.stringify({ msg: 'Hello from Node.js!', pid: process.pid }))
21 })
22
23 server.listen(3000, () => {
24   console.log(`Server started on port 3000`)
```

```
25 console.log(`Node version: ${process.version}`)
```

```
26 console.log(`Platform: ${process.platform}`)  
27 }
```

■ Interview Tip:

Node.js can handle 10,000+ concurrent connections with a single thread because it never waits for I/O — it delegates I/O to the OS/libuv and uses callbacks when done. Traditional servers (Apache) create a thread per connection — Node is far more memory-efficient.

Q2 Why is Node.js single-threaded? Is it really single-threaded?

■ Answer

Node.js JavaScript execution is single-threaded — only one JS operation runs at a time. BUT the underlying libuv is multi-threaded for heavy I/O operations.

What's single-threaded:

- Your JavaScript code
- V8 engine execution
- The event loop itself

What's multi-threaded (libuv thread pool):

- File system operations (fs module)
- DNS lookups (dns.lookup)
- Crypto operations (bcrypt, large hashes)
- User-defined C++ add-ons

What's OS-level async (not thread pool):

- Network I/O (TCP, UDP, HTTP requests)
- Unix domain sockets
- Pipes

Thread pool size:

- Default: 4 threads
- Configure: `UV_THREADPOOL_SIZE=16 node app.js`

```

1 // Demonstrating the 'single-threaded' nature
2 // And why CPU-intensive tasks are a problem
3
4 const http = require('http')
5
6 http.createServer((req, res) => {
7   if (req.url === '/slow') {
8     // ■ CPU-intensive – BLOCKS the event loop!
9     // While this runs, NO other requests can be handled
10    let result = 0
11    for (let i = 0; i < 5e9; i++) result += i // 5 billion iterations
12    res.end(`Result: ${result}`) // takes ~3 seconds – all others wait!
13   } else {
14     res.end('Fast response')
15   }
16 }).listen(3000)
17
18 // ■ Fix: offload CPU work to Worker Thread
19 const { Worker, isMainThread, parentPort, workerData } = require('worker_threads')
20
21 if (isMainThread) {
22   http.createServer((req, res) => {
23     if (req.url === '/slow') {

```

```

24     new Worker(__filename, { workerData: { n: 5e9 } })

```

```
25     .on('message', result => res.end(`Result: ${result}`))
26   } else {
27     res.end('Fast response') // still handles instantly!
28   }
29   }).listen(3000)
30 } else {
31   let result = 0
32   for (let i = 0; i < workerData.n; i++) result += i
33   parentPort.postMessage(result) // send result back
34 }
```

■ Interview Tip:

The CPU-blocking issue is the Achilles heel of Node.js. Any synchronous loop longer than ~50ms will cause visible latency spikes. Always offload heavy computation to Worker Threads, child processes, or a separate service.

Q3 What is the Event Loop? Explain all 6 phases

■ Answer

The Event Loop is the mechanism that allows Node.js to perform non-blocking I/O. It continuously checks if there's work to do and processes it phase by phase.

6 Event Loop Phases (in order):

- Phase 1: Timers — `setTimeout()` and `setInterval()` callbacks
- Phase 2: Pending I/O — I/O callbacks deferred from previous loop iteration
- Phase 3: Idle/Prepare — internal Node.js use only
- Phase 4: Poll — retrieve new I/O events; execute I/O callbacks

(block here if no other work pending)

- Phase 5: Check — `setImmediate()` callbacks
- Phase 6: Close Callbacks — close events (`socket.on('close', ...)`)

Between EVERY phase:

- `process.nextTick` queue is fully drained first
- Then Promise microtask queue is fully drained
- THEN the next phase begins

```

1 // COMPLETE event loop order demonstration
2 console.log('=== SYNC START ===')
3
4 // Macrotasks (Timers phase)
5 setTimeout(() => console.log('7: setTimeout(0)'), 0)
6 setTimeout(() => console.log('8: setTimeout(100)'), 100)
7
8 // Check phase
9 setImmediate(() => console.log('6: setImmediate'))
10
11 // Microtasks
12 Promise.resolve().then(() => {
13   console.log('3: Promise.then')
14   // Adding nextTick INSIDE promise:
15   process.nextTick(() => console.log('4: nextTick inside promise'))
16 })
17
18 // process.nextTick – HIGHEST priority async
19 process.nextTick(() => {
20   console.log('2: process.nextTick 1')
21   process.nextTick(() => console.log('5: nextTick inside nextTick'))
22 })
23
24 console.log('=== SYNC END ===')
25
26 // OUTPUT ORDER:
27 // === SYNC START ===           (synchronous)
28 // === SYNC END ===             (synchronous)
29 // 2: process.nextTick 1         (nextTick queue)

```

```

30 // 3: Promise.then              (microtask queue)

```

```
31 // 4: nextTick inside promise (nextTick inserted in microtask - runs after ALL current microtasks)
32 // 5: nextTick inside nextTick (nextTick from nextTick - runs immediately)
33 // 6: setImmediate           (Check phase)
34 // 7: setTimeout(0)          (Timers phase)
35 // 8: setTimeout(100)        (next Timers phase)
```

■ Interview Tip:

This exact output order question is asked in EVERY Node.js senior interview. Memorize: sync → nextTick → Promise → setImmediate → setTimeout(0). And remember: process.nextTick inside a Promise runs AFTER all current promises but BEFORE the next event loop phase.

Q4

What is non-blocking I/O? Blocking vs Non-blocking?

■ Answer

Non-blocking I/O means Node.js starts an I/O operation and immediately continues executing other code — it doesn't wait for the operation to complete.

Blocking I/O (bad in Node.js):

- Execution pauses until operation finishes
- Thread sits idle wasting resources
- Other requests queue up and wait
- OK only during startup (loading config)

Non-blocking I/O (good):

- Operation starts in background
- Callback registered for completion
- Thread immediately handles next request
- Callback fires when I/O completes

Why Node.js can handle 10K+ connections:

- Thread never blocks waiting for I/O
- OS handles actual I/O asynchronously
- Thread dispatches I/O and moves on
- Callback called when OS signals completion

```
1  const fs = require('fs')
2  const { readFile, writeFile } = require('fs/promises')
3
4  // ■ BLOCKING – halts event loop completely!
5  const data = fs.readFileSync('large-file.json') // thread STOPS here
6  console.log('This waits for file read to finish')
7
8  // ■ NON-BLOCKING callback style
9  fs.readFile('large-file.json', 'utf8', (err, data) => {
10     if (err) throw err
11     console.log('File read done:', data.length, 'chars')
12 })
13 console.log('This runs IMMEDIATELY – before file read!') // runs first!
14
15 // ■ NON-BLOCKING with async/await (cleaner)
16 async function processFile() {
17     const data = await readFile('large-file.json', 'utf8') // non-blocking await
18     const processed = data.toUpperCase()
19     await writeFile('output.json', processed)
20     return 'done'
21 }
22
23 // Running multiple I/O operations in PARALLEL (fastest!)
24 async function loadAllData() {
25     const [config, users, products] = await Promise.all([
```

```
26     readFile('config.json', 'utf8'), // all three start simultaneously!
```



```
27     readFile('users.json', 'utf8'),    // not waiting for each other
28     readFile('products.json', 'utf8')
29   ])
30   return { config: JSON.parse(config), users: JSON.parse(users), products: JSON.parse(products) }
31 }
32 // Total time ≈ max(time1, time2, time3) instead of time1+time2+time3
```

■ Interview Tip:

Rule: NEVER use sync APIs (readFileSync, writeFileSync) inside request handlers or any hot code path. They block the entire event loop. Only use sync APIs during application startup (loading config before server starts).

Q5 What is process.nextTick vs setImmediate vs setTimeout?

■ Answer

Three ways to defer execution asynchronously, but with very different priorities.

process.nextTick(fn):

- Runs BEFORE the event loop continues to NEXT phase
- Highest priority async in Node.js
- NOT part of event loop phases — runs between ANY two operations
- Both sync queue AND between event loop phases
- Can starve I/O if used in infinite recursion!

setImmediate(fn):

- Runs in CHECK phase — after Poll phase
- After I/O callbacks in same loop iteration
- Predictable: always runs after I/O in same tick

setTimeout(fn, 0):

- Runs in TIMERS phase
- Minimum delay ~1ms (not truly 0)
- Can be non-deterministic vs setImmediate at top level

Priority: nextTick > Promise > setImmediate > setTimeout

```

1 // Priority demonstration
2 console.log('start')
3
4 setTimeout(() => console.log('A: setTimeout 0'), 0)
5 setImmediate(() => console.log('B: setImmediate'))
6 Promise.resolve().then(() => console.log('C: Promise'))
7 process.nextTick(() => console.log('D: nextTick'))
8
9 console.log('end')
10 // Output: start → end → D → C → B → A
11
12 // INSIDE I/O callback: setImmediate always before setTimeout
13 const fs = require('fs')
14 fs.readFile(__filename, () => {
15   setTimeout(() => console.log('timeout')) // unpredictable outside I/O
16   setImmediate(() => console.log('immediate')) // ALWAYS first inside I/O
17 })
18
19 // REAL USE CASES:
20
21 // nextTick: emit events after current call returns
22 class EventEmitter {
23   emit(event, ...args) {
24     process.nextTick(() => { // fires AFTER current function returns
25       this.listeners(event).forEach(fn => fn(...args))
26     })
  }
}

```

```

27 }
28 }
29
30 // setImmediate: break up CPU work, yield to I/O
31 function processInChunks(items, i = 0) {
32   if (i >= items.length) return
33   processItem(items[i])
34   setImmediate(() => processInChunks(items, i + 1))
35   // yields to I/O between each item!
36 }
37
38 // process.nextTick: API consistency
39 function getUser(id, callback) {
40   const cached = cache.get(id)
41   if (cached) {
42     process.nextTick(() => callback(null, cached)) // always async!
43     // Without nextTick, sometimes sync/sometimes async = BUG!
44     return
45   }
46   db.find(id, callback)
47 }

```

■ Interview Tip:

Recursive `process.nextTick` can starve I/O callbacks — the `nextTick` queue keeps adding to itself and I/O callbacks never run. Use `setImmediate` for yielding to I/O. The 'always async' pattern with `nextTick` prevents Zalgo (mixed sync/async callbacks).

Q6 What is CommonJS (require) vs ES Modules (import)?

■ Answer

Node.js has two module systems: CommonJS (original) and ES Modules (modern standard).

CommonJS (CJS):

- `require()` — synchronous, loads module immediately
- `module.exports` / `exports` — export values
- Dynamic: `require()` inside if blocks, functions
- `__dirname`, `__filename` available by default
- Works in all Node.js versions
- Default for `.js` files (unless `type:module` in `package.json`)

ES Modules (ESM):

- `import/export` — static, analyzed before execution
- Top-level `await` supported
- Tree-shakeable — bundlers remove unused exports
- Use `.mjs` extension OR `type:module` in `package.json`
- `__dirname` not available (use `import.meta.url`)

Key differences:

- CJS: synchronous loading | ESM: async loading
- CJS: dynamic imports | ESM: static by default
- CJS: copies values | ESM: live bindings (reflect updates)

```

1  // ■■ COMMONJS ■■
2  // math.js
3  const PI = 3.14159
4  function add(a, b) { return a + b }
5  function multiply(a, b) { return a * b }
6
7  module.exports = { PI, add, multiply }
8  // OR short form:
9  exports.divide = (a, b) => a / b
10
11 // main.js – require is synchronous
12 const { add, PI } = require('./math') // .js extension optional
13 const path = require('path')         // built-in
14 const express = require('express')    // npm package
15
16 // Dynamic require (only in CJS!)
17 const feature = process.env.FEATURE
18 const handler = require(`./handlers/${feature}`) // runtime path!
19
20 // ■■ ES MODULES ■■
21 // math.mjs OR package.json: { 'type': 'module' }
22 export const PI = 3.14159
23 export function add(a, b) { return a + b }
24 export default class Calculator { } // default export

```

```

26 // main.mjs
27 import { add, PI } from './math.mjs' // named imports
28 import Calculator from './math.mjs' // default import
29 import * as math from './math.mjs' // namespace import
30 import './side-effects.mjs' // side-effect only
31
32 // Dynamic import in ESM (returns Promise)
33 const { add: sum } = await import('./math.mjs')
34
35 // ESM: get __dirname equivalent
36 import { fileURLToPath } from 'url'
37 import { dirname, join } from 'path'
38 const __filename = fileURLToPath(import.meta.url)
39 const __dirname = dirname(__filename)
40 const configPath = join(__dirname, 'config.json')

```

■ Interview Tip:

Most production Node.js apps still use CommonJS because the npm ecosystem was built around it. New projects should use ESM for future compatibility. Mixing is complex — you can import CJS from ESM but not vice versa.

What is the process object? All important properties?

■ Answer

process is a global object in Node.js providing info and control over the current process. It's available everywhere — no require() needed.

Key properties:

- process.env — all environment variables (object)
- process.argv — command line arguments array
- process.pid — current process ID (number)
- process.ppid — parent process ID
- process.version — Node.js version string
- process.platform — OS: 'linux', 'win32', 'darwin'
- process.arch — CPU: 'x64', 'arm64'
- process.cwd() — current working directory
- process.memoryUsage() — memory stats
- process.cpuUsage() — CPU stats
- process.uptime() — seconds since process started

Key methods:

- process.exit(code) — terminate (0=success, 1=error)
- process.nextTick(fn) — defer execution
- process.on(event, fn) — listen to events
- process.kill(pid, signal) — send signal to process

```

1 // Environment variables — most common use
2 const PORT = parseInt(process.env.PORT) || 3000
3 const NODE_ENV = process.env.NODE_ENV || 'development'
4 const DB_URL = process.env.DATABASE_URL
5 const isDev = NODE_ENV === 'development'
6 const isProd = NODE_ENV === 'production'
7
8 // Command line args: node app.js --port 4000 --debug
9 console.log(process.argv)
10 // ['node', '/path/app.js', '--port', '4000', '--debug']
11 const args = Object.fromEntries(
12   process.argv.slice(2).reduce((acc, arg, i, arr) => {
13     if (arg.startsWith('--')) acc.push([arg.slice(2), arr[i+1]])
14     return acc
15   }, [])
16 )
17 // { port: '4000', debug: undefined }
18
19 // Memory monitoring
20 const used = process.memoryUsage()
21 console.log({
22   heapUsed: `${Math.round(used.heapUsed/1024/1024)}MB`, // used heap
23   heapTotal: `${Math.round(used.heapTotal/1024/1024)}MB`, // total heap

```

```

24   rss: `${Math.round(used.rss/1024/1024)}MB`, // resident set size

```

```
25   external: `${Math.round(used.external/1024/1024)}MB`    // C++ objects
26 })
27
28 // Process events – handle gracefully
29 process.on('uncaughtException', (err) => {
30   console.error('UNCAUGHT:', err.message)
31   process.exit(1)    // restart via PM2
32 })
33 process.on('unhandledRejection', (reason, promise) => {
34   console.error('UNHANDLED REJECTION:', reason)
35   process.exit(1)
36 })
37 process.on('SIGTERM', gracefulShutdown)    // kill command, K8s
38 process.on('SIGINT', gracefulShutdown)    // Ctrl+C
```

■ Interview Tip:

Never expose process.env values to clients. process.env.DATABASE_URL contains credentials. Use a config module to validate and sanitize environment variables at startup using joi or zod.

What is the `__dirname` and `__filename`?

■ Answer

`__dirname` is the absolute path of the DIRECTORY containing the current file.

`__filename` is the absolute path of the current file itself.

Why they're important:

- Relative paths in `require()` are fine for imports
- But `fs.readFile('./config.json')` is relative to CWD (current working directory)
- CWD changes when you run the app from different directories!
- `__dirname` is always relative to the FILE — safe to use

In CommonJS:

- `__dirname` and `__filename` are automatically available

In ES Modules:

- NOT available — must create them from `import.meta.url`

path module utilities:

- `path.join()` — joins paths with OS separator
- `path.resolve()` — creates absolute path
- `path.dirname()` — get directory from file path
- `path.basename()` — get filename
- `path.extname()` — get extension

```

1  const path = require('path')
2  const fs = require('fs')
3
4  // __dirname = '/Users/kaushal/projects/myapp/src'
5  // __filename = '/Users/kaushal/projects/myapp/src/app.js'
6
7  // ■ DANGEROUS — relative to CWD, not file location
8  const config = fs.readFileSync('./config.json') // breaks if run from /
9
10 // ■ SAFE — always relative to this file's location
11 const config2 = fs.readFileSync(path.join(__dirname, 'config.json'))
12 const config3 = fs.readFileSync(path.resolve(__dirname, '..', 'config.json'))
13
14 // path.join vs path.resolve
15 path.join('/a', 'b', 'c') // '/a/b/c'
16 path.join('/a', '../b', 'c') // '/b/c' (resolves ..)
17 path.resolve('/a', 'b') // '/a/b' (absolute path)
18 path.resolve('relative') // adds CWD prefix
19
20 // Path utilities
21 path.dirname('/home/kaushal/app.js') // '/home/kaushal'
22 path.basename('/home/kaushal/app.js') // 'app.js'
23 path.basename('/home/kaushal/app.js', '.js') // 'app'
24 path.extname('/home/kaushal/app.js') // '.js'

```



```
26 // ESM equivalent (for .mjs files)
27 import { fileURLToPath } from 'url'
28 import { dirname, join } from 'path'
29 const __filename = fileURLToPath(import.meta.url)
30 const __dirname = dirname(__filename)
31 const configPath = join(__dirname, 'config.json') // now safe!
```

■ Interview Tip:

Always use `path.join(__dirname, 'filename')` for file paths in your Node.js modules. This is one of the most common Node.js bugs — files work locally but break in production because the working directory is different.

What is the fs module? Sync vs Async operations?

■ Answer

The fs (file system) module provides APIs for reading, writing, and managing files. Every method has synchronous AND asynchronous versions.

RULE: Never use sync in production request handlers

- readFileSync, writeFileSync, existsSync — block the event loop!
- Sync only OK during startup (before server starts listening)

Three async styles:

- Callbacks — original style: fs.readFile(path, callback)
- Promises — fs/promises: import { readFile } from 'fs/promises'
- Streams — for large files: fs.createReadStream(path)

Key operations:

- readFile/writeFile — read/write entire file
- appendFile — append without overwrite
- unlink — delete file
- mkdir/rmdir — create/delete directory
- readdir — list directory contents
- stat — file metadata (size, dates, isFile, isDirectory)
- rename — rename/move file
- copyFile — copy file

```
1  const fs = require('fs')
2  const { readFile, writeFile, mkdir, readdir, stat, unlink } = require('fs/promises')
3
4  // ■ Async with Promises (recommended for all new code)
5  async function fileOperations() {
6    // Read file
7    const json = await readFile('./config.json', 'utf8')
8    const config = JSON.parse(json)
9
10   // Write file
11   await writeFile('./output.txt', 'Hello World!', 'utf8')
12
13   // Append to file (doesn't overwrite)
14   const { appendFile } = require('fs/promises')
15   await appendFile('./log.txt', `\n${new Date()}: Request processed`)
16
17   // Create directory recursively
18   await mkdir('./uploads/2024/images', { recursive: true })
19
20   // List directory contents
21   const files = await readdir('./uploads')
22   console.log('Files:', files)
23
24   // Get file stats
```

```
25  const info = await stat('./output.txt')
```

```

26 console.log({
27   size: info.size,
28   isFile: info.isFile(),
29   isDir: info.isDirectory(),
30   created: info.birthtime,
31   modified: info.mtime
32 })
33
34 // Check if file exists (safe way)
35 try {
36   await stat('./maybe-exists.txt')
37   console.log('File exists!')
38 } catch (e) {
39   if (e.code === 'ENOENT') console.log('File does not exist')
40   else throw e
41 }
42
43 // Delete file
44 await unlink('./temp.txt')
45 }
46
47 // ■ Only OK at startup (before server.listen)
48 const config = JSON.parse(fs.readFileSync('./config.json', 'utf8'))
49 console.log('Config loaded:', config.appName)

```

■ Interview Tip:

`fs.existsSync()` is a common mistake — it's synchronous and blocks the event loop. Instead, use `try/catch` with `await stat()` to check if a file exists. This is the recommended pattern for async file existence checks.

What are Streams in Node.js? Types and use cases?

■ Answer

Streams process data piece-by-piece (chunks) instead of loading it all into memory. Essential for handling large files, HTTP responses, and real-time data.

4 Types of Streams:

- Readable — data source: `fs.createReadStream`, `http.IncomingMessage`, `process.stdin`
- Writable — data destination: `fs.createWriteStream`, `http.ServerResponse`, `process.stdout`
- Duplex — both readable AND writable: `net.Socket`, `WebSocket`
- Transform — duplex that modifies data: `zlib.createGzip`, `crypto.createCipher`

Why use streams:

- Memory efficiency — process 10GB file with 50MB RAM
- Time efficiency — start processing before all data arrives
- Composable — pipe streams together like Unix commands

Stream events:

- 'data' — chunk of data available
- 'end' — no more data (readable)
- 'finish' — all data flushed (writable)
- 'error' — error occurred
- 'close' — stream fully closed

```

1  const fs = require('fs')
2  const zlib = require('zlib')
3  const { Transform, pipeline } = require('stream')
4  const { promisify } = require('util')
5  const pipelineAsync = promisify(pipeline) // promise version!
6
7  // Basic: pipe readable to writable
8  fs.createReadStream('input.txt')
9    .pipe(fs.createWriteStream('output.txt'))
10
11 // Chain transforms: read → gzip compress → write
12 await pipelineAsync(
13   fs.createReadStream('huge-file.log'), // reads in 64KB chunks
14   zlib.createGzip(), // compresses each chunk
15   fs.createWriteStream('archive.log.gz') // writes compressed
16 ) // Uses ~50MB RAM for a 5GB file!
17
18 // Custom Transform stream
19 const csvToJson = new Transform({
20   objectMode: true,
21   transform(chunk, encoding, callback) {
22     const line = chunk.toString().trim()
23     if (!line) return callback()
24     const [name, email, age] = line.split(',')
25     this.push(JSON.stringify({ name, email, age: parseInt(age) }) + '\n')
26   },
27   callback()

```

```

27   }
28 })
29
30 // Stream a large file as HTTP response
31 const http = require('http')
32 http.createServer((req, res) => {
33   res.setHeader('Content-Type', 'application/octet-stream')
34   res.setHeader('Content-Disposition', 'attachment; filename=data.csv')
35   fs.createReadStream('./exports/large-data.csv')
36     .on('error', (err) => res.destroy(err))
37     .pipe(res) // stream directly to response – user gets data immediately!
38 }).listen(3000)
39
40 // Read stream events manually
41 const readStream = fs.createReadStream('data.txt', { highWaterMark: 64*1024 })
42 let totalBytes = 0
43 readStream.on('data', chunk => { totalBytes += chunk.length })
44 readStream.on('end', () => console.log(`Total: ${totalBytes} bytes`))
45 readStream.on('error', err => console.error('Stream error:', err))

```

■ Interview Tip:

Always use `pipeline()` instead of `.pipe()` for production code. `pipeline()` properly handles backpressure and cleans up streams on error. With `.pipe()`, if one stream errors, the others aren't destroyed — causing memory leaks.

What is Backpressure in streams?

■ Answer

Backpressure occurs when a writable stream can't process data as fast as a readable produces it. Without backpressure handling, the buffer overflows and crashes the process.

The problem:

- Readable stream produces data at 100MB/s
- Writable stream processes at 10MB/s
- 90MB/s accumulates in memory buffer
- Eventually: out of memory error!

How pipe() handles backpressure:

- `writable.write()` returns false when buffer full
- `pipe()` pauses the readable automatically
- When buffer drains, 'drain' event fires
- `pipe()` resumes the readable

Using pipeline():

- Handles ALL of this automatically
- Also properly handles errors and cleanup
- Recommended for all stream composition

```
1  const fs = require('fs')
2  const { pipeline } = require('stream')
3  const { promisify } = require('util')
4  const pipe = promisify(pipeline)
5
6  // ■ Manual backpressure – complex and error-prone
7  const readable = fs.createReadStream('huge.csv')
8  const writable = fs.createWriteStream('out.csv')
9
10 readable.on('data', chunk => {
11   const canContinue = writable.write(chunk) // returns false when buffer full!
12   if (!canContinue) {
13     readable.pause() // stop producing!
14     writable.once('drain', () => readable.resume()) // wait for buffer to drain
15   }
16 })
17 readable.on('end', () => writable.end())
18 readable.on('error', err => writable.destroy(err))
19 writable.on('error', err => readable.destroy(err))
20
21 // ■ pipeline() handles ALL of the above automatically!
22 await pipe(
23   fs.createReadStream('huge.csv'),
24   fs.createWriteStream('out.csv')
25 )
26
```

```
27 // highWaterMark – controls buffer size
```

```
28 const fast = fs.createReadStream('data', { highWaterMark: 16 * 1024 }) // 16KB chunks
29 const slow = fs.createWriteStream('out', { highWaterMark: 1024 }) // 1KB buffer
30 // pipeline handles the backpressure between them
31 await pipe(fast, slow)
32
33 // Check writable buffer state
34 const ws = fs.createWriteStream('test.txt')
35 ws.write('first chunk') // returns true – buffer not full
36 // Keep writing until false:
37 const full = !ws.write('big chunk') // might return false
38 if (full) {
39   console.log('Buffer full! Waiting for drain...')
40   ws.once('drain', () => console.log('Buffer drained, continue writing'))
41 }
```

■ Interview Tip:

Backpressure is why you should always use `pipeline()` over manual `.pipe()` chains. Netflix's video streaming, S3 uploads, and large file exports all rely on proper backpressure handling — without it, the server runs out of memory.

What is the Buffer class in Node.js?

■ Answer

Buffer represents raw binary data — a fixed-size chunk of memory outside V8 heap. Used for: binary files, network data, crypto operations, encoding conversions.

Creating Buffers:

- `Buffer.alloc(size)` — zero-filled buffer (safe)
- `Buffer.allocUnsafe(size)` — uninitialized, faster but may contain old data
- `Buffer.from(string, encoding)` — from string
- `Buffer.from(array)` — from array of bytes
- `Buffer.from(arrayBuffer)` — from `ArrayBuffer`

Encodings:

- `'utf8'` — Unicode text (default)
- `'base64'` — base64 encoded string
- `'hex'` — hexadecimal
- `'binary'` — raw binary
- `'ascii'` — ASCII-only

Buffer in Streams:

- Stream chunks are Buffers by default
- Use `objectMode:true` for non-Buffer streams

```
1  const { Buffer } = require('buffer') // available globally
2
3  // Creating buffers
4  const buf1 = Buffer.alloc(10)         // <Buffer 00 00 00 00 ...>
5  const buf2 = Buffer.from('Hello!')    // from string (UTF-8 default)
6  const buf3 = Buffer.from([0x48, 0x65, 0x6c, 0x6c, 0x6f]) // 'Hello'
7
8  // Buffer to string conversions
9  buf2.toString()                      // 'Hello!' (UTF-8 default)
10 buf2.toString('base64')              // 'SGVsbG8h'
11 buf2.toString('hex')                 // '48656c6c6f21'
12
13 // Base64 encode/decode (used in JWTs, file uploads, emails)
14 const encoded = Buffer.from('username:password').toString('base64')
15 // 'dXNlcm5hbWU6cGFzc3dvcmQ='
16 const decoded = Buffer.from(encoded, 'base64').toString('utf8')
17 // 'username:password'
18
19 // Reading binary files as Buffer
20 const fs = require('fs')
21 const imageBuffer = fs.readFileSync('./photo.jpg') // Buffer
22 console.log(`Image size: ${imageBuffer.length} bytes`)
23 const base64Image = imageBuffer.toString('base64')
24 // Use in HTML: <img src='data:image/jpeg;base64,${base64Image}'>
25
```

```
26 // Buffer comparison
```



```
27 const a = Buffer.from('Hello')
28 const b = Buffer.from('Hello')
29 a === b           // false (different objects!)
30 a.equals(b)       // true (same content)
31 Buffer.compare(a, b) // 0 (equal), <0 (a<b), >0 (a>b)
32
33 // Concatenate buffers
34 const full = Buffer.concat([buf2, buf3], buf2.length + buf3.length)
35
36 // Buffer.allocUnsafe – faster but MUST be filled before use!
37 const fast = Buffer.allocUnsafe(100) // may have garbage data
38 fast.fill(0) // ← always zero-fill if you'll read before writing
```

■ Interview Tip:

Buffer.allocUnsafe() is 3-4x faster than Buffer.alloc() but contains arbitrary memory data (whatever was at that address before). Never read allocUnsafe buffer without filling it first — security risk if that data could reach the network.

What is the path module? Key methods?

■ Answer

The path module provides utilities for working with file and directory paths. Handles OS differences (Windows uses \, Unix uses /) automatically.

Most important methods:

- `path.join(...parts)` — join path segments with OS separator
- `path.resolve(...parts)` — build absolute path from right to left
- `path.dirname(filePath)` — directory name
- `path.basename(filePath, ext?)` — file name
- `path.extname(filePath)` — file extension
- `path.parse(filePath)` — {root, dir, base, name, ext}
- `path.format(pathObj)` — opposite of parse
- `path.isAbsolute(path)` — check if absolute
- `path.relative(from, to)` — relative path between two paths

path.sep and path.delimiter:

- `path.sep` — '/' (Unix) or '\\' (Windows)
- `path.delimiter` — ':' (Unix) or ';' (Windows PATH)

```

1  const path = require('path')
2
3  // path.join — most common, handles OS differences
4  path.join('/home', 'kaushal', 'app.js') // '/home/kaushal/app.js'
5  path.join('/home', '../etc', 'config')   // '/etc/config' (resolves ..)
6  path.join('a', 'b', 'c')                 // 'a/b/c' (Unix) or 'a\\b\\c' (Windows)
7
8  // path.resolve — builds absolute path
9  path.resolve('foo', 'bar')                // '/cwd/foo/bar'
10 path.resolve('/a', 'b')                   // '/a/b'
11 path.resolve('/a', '/b', 'c')             // '/b/c' (absolute path resets)
12
13 // path.dirname and basename
14 path.dirname('/home/kaushal/app.js')      // '/home/kaushal'
15 path.basename('/home/kaushal/app.js')    // 'app.js'
16 path.basename('/home/kaushal/app.js', '.js') // 'app'
17 path.extname('/home/kaushal/app.js')     // '.js'
18 path.extname('/home/kaushal/README')     // ''
19
20 // path.parse — break into parts
21 path.parse('/home/kaushal/app.js')
22 // { root: '/', dir: '/home/kaushal', base: 'app.js', name: 'app', ext: '.js' }
23
24 // Real-world: safe file paths in Express
25 const express = require('express')
26 const app = express()
27 app.get('/download/:filename', (req, res) => {
28   // ■ DANGEROUS — path traversal attack!

```

```

29 // req.params.filename could be '../../etc/passwd'

```

```
30  const badPath = path.join('./uploads', req.params.filename)
31
32  // ■ SAFE – sanitize and validate
33  const safeName = path.basename(req.params.filename) // remove any path
34  const safePath = path.join(__dirname, 'uploads', safeName)
35  // verify it's still inside uploads directory!
36  if (!safePath.startsWith(path.join(__dirname, 'uploads'))) {
37    return res.status(403).json({ error: 'Access denied' })
38  }
39  res.sendFile(safePath)
40  })
```

■ Interview Tip:

`path.basename()` is a crucial security tool — it strips directory components from a filename. If a user uploads 'malware/../../etc/passwd', `path.basename()` returns 'passwd', and your `path.join(__dirname, 'uploads', 'passwd')` stays within uploads.

What is the http module? How to create HTTP server?

■ Answer

The http module provides low-level HTTP functionality in Node.js. Express is built on top of it — understanding http helps you understand Express.

http.createServer((req, res) => {}):

- req — IncomingMessage — readable stream with request data
- res — ServerResponse — writable stream for sending response

Request object key properties:

- req.method — 'GET', 'POST', 'PUT', etc.
- req.url — '/path?query=value'
- req.headers — object with all headers
- req.socket.remoteAddress — client IP

Response methods:

- res.writeHead(statusCode, headers) — set status and headers
- res.setHeader(name, value) — set single header
- res.write(chunk) — write chunk of body
- res.end(data?) — finish response
- res.statusCode = 200 — set status code

```

1  const http = require('http')
2  const url = require('url')
3
4  // HTTP server from scratch (what Express builds on)
5  const server = http.createServer(async (req, res) => {
6    const parsedUrl = url.parse(req.url, true) // parse query string
7    const pathname = parsedUrl.pathname
8
9    // Set CORS headers
10   res.setHeader('Access-Control-Allow-Origin', '*')
11   res.setHeader('Content-Type', 'application/json')
12
13   // Manual routing
14   if (pathname === '/api/health' && req.method === 'GET') {
15     res.writeHead(200)
16     res.end(JSON.stringify({ status: 'ok', uptime: process.uptime() }))
17
18   } else if (pathname === '/api/users' && req.method === 'POST') {
19     // Read request body (it's a stream!)
20     let body = ''
21     req.on('data', chunk => body += chunk.toString())
22     req.on('end', () => {
23       try {
24         const data = JSON.parse(body)
25         res.writeHead(201)
26         res.end(JSON.stringify({ user: data, id: Date.now() }))
27
28       } catch {

```

```

28     res.writeHead(400)
29     res.end(JSON.stringify({ error: 'Invalid JSON' }))
30   }
31 })
32
33 } else {
34   res.writeHead(404)
35   res.end(JSON.stringify({ error: 'Not found' }))
36 }
37 })
38
39 server.listen(3000, '0.0.0.0', () => {
40   console.log('Server listening on port 3000')
41 })
42
43 server.on('error', (err) => {
44   if (err.code === 'EADDRINUSE') {
45     console.error('Port 3000 already in use!')
46     process.exit(1)
47   }
48 })

```

■ Interview Tip:

The request body is a stream — you must listen to 'data' events to collect it. Express's `body-parser/express.json()` does this for you. Understanding the raw http module helps when debugging why a request body is missing.

Q15 What is the url module? URL parsing in Node.js?

■ Answer

Node.js provides two URL APIs: the legacy url module and the modern WHATWG URL API.

WHATWG URL API (modern — recommended):

- Available as global URL in Node.js 10+
- Also: import { URL, URLSearchParams } from 'url'
- Follows the browser's URL standard

Legacy url.parse() (deprecated):

- Returns object with protocol, hostname, path, query, etc.
- Still used in older codebases

URLSearchParams:

- Parse and manipulate query strings
- Available globally or via URLSearchParams

querystring module (legacy):

- querystring.parse('a=1&b=2') → { a: '1', b: '2' }
- Replaced by URLSearchParams

```
1  const { URL, URLSearchParams } = require('url')
2
3  // WHATWG URL (modern)
4  const myUrl = new URL('https://api.example.com/users?page=2&limit=10&sort=name')
5
6  console.log(myUrl.protocol) // 'https:'
7  console.log(myUrl.hostname) // 'api.example.com'
8  console.log(myUrl.pathname) // '/users'
9  console.log(myUrl.search)   // '?page=2&limit=10&sort=name'
10 console.log(myUrl.href)     // full URL string
11
12 // URLSearchParams – read query string
13 const params = myUrl.searchParams
14 params.get('page')          // '2'
15 params.get('limit')         // '10'
16 params.has('sort')          // true
17 params.getAll('tag')        // [] (multiple values!)
18
19 // Build URL with params
20 const apiUrl = new URL('https://api.example.com/products')
21 apiUrl.searchParams.set('category', 'electronics')
22 apiUrl.searchParams.set('page', '1')
23 apiUrl.searchParams.append('tag', 'sale')
24 apiUrl.searchParams.append('tag', 'new')
25 console.log(apiUrl.toString())
26 // 'https://api.example.com/products?category=electronics&page=1&tag=sale&tag=new'
27
```

```
28 // In Express – parse query string from request
```

```
29 app.get('/products', (req, res) => {
30   // Express already parses req.query
31   const { page = '1', limit = '20', category } = req.query
32
33   // Build upstream API URL
34   const upstreamUrl = new URL(`${process.env.API_BASE}/products`)
35   upstreamUrl.searchParams.set('page', page)
36   upstreamUrl.searchParams.set('limit', limit)
37   if (category) upstreamUrl.searchParams.set('category', category)
38
39   fetch(upstreamUrl.toString()).then(r => r.json()).then(data => res.json(data))
40 })
```

■ Interview Tip:

Use the WHATWG URL API instead of the legacy `url.parse()`. It matches the browser API, handles edge cases correctly, and properly encodes special characters in query parameters.

Q16 What is the crypto module? Common operations?

■ Answer

The crypto module provides cryptographic functionality: hashing, HMAC, encryption, random bytes.

Common operations:

- Hashing — one-way: SHA-256, MD5 (for checksums)
- HMAC — keyed hash: verify message authenticity
- Random bytes — cryptographically secure random
- Encryption — AES, RSA (symmetric/asymmetric)
- Key derivation — PBKDF2, scrypt (for password hashing)

When to use built-in crypto vs libraries:

- Passwords: use bcrypt or argon2 (safer than crypto.pbkdf2)
- JWTs: use jsonwebtoken library
- Random IDs: crypto.randomUUID() (built-in, v14.17+)
- File checksums: crypto.createHash('sha256')
- API keys: crypto.randomBytes(32).toString('hex')
- HMACs for webhooks: crypto.createHmac

```

1  const crypto = require('crypto')
2
3  // Hashing (one-way)
4  const hash = crypto.createHash('sha256')
5  hash.update('Hello World')
6  const digest = hash.digest('hex') // '64ec88ca...'
7  // Or in one line:
8  const sha256 = crypto.createHash('sha256').update('data').digest('hex')
9
10 // File integrity check
11 const fs = require('fs')
12 const fileChecksum = () => {
13   const hash = crypto.createHash('sha256')
14   const stream = fs.createReadStream('./large-file.zip')
15   return new Promise((resolve, reject) => {
16     stream.on('data', d => hash.update(d))
17     stream.on('end', () => resolve(hash.digest('hex')))
18     stream.on('error', reject)
19   })
20 }
21
22 // Cryptographically secure random values
23 crypto.randomBytes(32).toString('hex') // 64-char hex string (for secrets)
24 crypto.randomBytes(16).toString('base64') // 24-char base64
25 crypto.randomUUID() // '9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d'
26
27 // Generate API key
28 const generateApiKey = () => `sk_${crypto.randomBytes(24).toString('base64url')}`
29

```

```

30 // HMAC — verify webhook signatures (Stripe, GitHub use this!)

```



```

31 const verifyWebhookSignature = (payload, signature, secret) => {
32   const expected = crypto
33     .createHmac('sha256', secret)
34     .update(payload) // raw body string, not parsed!
35     .digest('hex')
36   // Timing-safe comparison!
37   return crypto.timingSafeEqual(
38     Buffer.from(signature, 'hex'),
39     Buffer.from(expected, 'hex')
40   )
41 }
42
43 // AES-256 symmetric encryption
44 const key = crypto.randomBytes(32) // 256-bit key
45 const iv = crypto.randomBytes(16) // initialization vector
46 const cipher = crypto.createCipheriv('aes-256-cbc', key, iv)
47 let encrypted = cipher.update('sensitive data', 'utf8', 'hex')
48 encrypted += cipher.final('hex')

```

■ Interview Tip:

Always use `crypto.timingSafeEqual()` for comparing secrets (API keys, HMAC signatures). Regular `===` comparison leaks timing information — an attacker can measure response time to determine if their guess is getting 'closer' to the secret.

What is the os module?

■ Answer

The os module provides operating system-related utilities.

Useful for: system info, configuring workers, writing cross-platform code.

Key properties:

- `os.cpus()` — array of CPU info objects
- `os.totalmem()` — total system RAM in bytes
- `os.freemem()` — free system RAM in bytes
- `os.platform()` — 'linux', 'darwin', 'win32'
- `os.arch()` — 'x64', 'arm64'
- `os.hostname()` — machine hostname
- `os.homedir()` — user home directory
- `os.tmpdir()` — temp directory path
- `os.uptime()` — system uptime in seconds
- `os.EOL` — end-of-line character ('\n' or '\r\n')

```

1  const os = require('os')
2  const cluster = require('cluster')
3
4  // System information
5  console.log({
6    platform: os.platform(),           // 'linux'
7    arch: os.arch(),                   // 'x64'
8    cpus: os.cpus().length,             // 8 (number of CPU cores)
9    totalMem: `${Math.round(os.totalmem() / 1024**3)}GB`,
10   freeMem: `${Math.round(os.freemem() / 1024**3)}GB`,
11   hostname: os.hostname(),            // 'my-server'
12   uptime: `${Math.round(os.uptime() / 3600)} hours`,
13   tmpDir: os.tmpdir()                  // '/tmp'
14 })
15
16 // Practical: auto-configure workers based on CPU count
17 const numWorkers = os.cpus().length // use all available cores
18 console.log(`Starting ${numWorkers} workers for ${os.cpus()[0].model}`)
19
20 if (cluster.isMaster) {
21   for (let i = 0; i < numWorkers; i++) cluster.fork()
22 } else {
23   require('./server')
24 }
25
26 // Health check endpoint
27 app.get('/health', (req, res) => {
28   const memUsage = process.memoryUsage()
29   res.json({
30     status: 'ok',
31     timestamp: new Date().toISOString(),

```

```

32   uptime: `${process.uptime().toFixed(0)}s`,

```

```
33 memory: {
34   heapUsed: `${Math.round(memUsage.heapUsed/1024/1024)}MB`,
35   heapTotal: `${Math.round(memUsage.heapTotal/1024/1024)}MB`
36 },
37 system: {
38   cpus: os.cpus().length,
39   freeMemory: `${Math.round(os.freemem()/1024/1024)}MB`
40 },
41 pid: process.pid
42 })
43 })
```

■ Interview Tip:

`os.cpus().length` is how you dynamically configure cluster workers without hardcoding the number. A server with 4 cores starts 4 workers, a server with 16 cores starts 16. Your code adapts to the hardware automatically.

What are child processes in Node.js?

■ Answer

Child processes allow Node.js to spawn new processes to run system commands or other scripts. Essential for: running shell commands, spawning Python/other languages, parallelizing CPU work.

4 ways to create child processes:

- `exec(cmd, callback)` — run shell command, buffer output
- `execFile(file, args, callback)` — run executable, no shell
- `spawn(cmd, args)` — streaming I/O, for large output
- `fork(modulePath)` — spawn another Node.js process with IPC

exec vs spawn:

- `exec`: buffers all output in memory — good for small commands
- `spawn`: streams output — good for large output or long-running processes

fork vs cluster:

- `fork`: general purpose IPC between parent and child Node processes
- `cluster`: specifically for multiple instances of HTTP server

```

1  const { exec, spawn, fork, execFile } = require('child_process')
2  const { promisify } = require('util')
3  const execAsync = promisify(exec)
4
5  // exec — run shell command, get result as string
6  exec('ls -la /tmp', (err, stdout, stderr) => {
7    if (err) console.error(err)
8    console.log(stdout)
9  })
10
11 // exec with promises — cleaner
12 const { stdout } = await execAsync('git log --oneline -5')
13 console.log('Recent commits:', stdout)
14
15 // spawn — streaming output for large commands
16 const child = spawn('find', ['/', '-name', '*.log'], { stdio: 'pipe' })
17 child.stdout.on('data', data => process.stdout.write(data))
18 child.stderr.on('data', data => process.stderr.write(data))
19 child.on('close', code => console.log(`Done with code ${code}`))
20
21 // fork — spawn child Node.js with IPC (Inter-Process Communication)
22 // worker.js
23 // process.on('message', ({ data }) => {
24 //   const result = heavyCompute(data)
25 //   process.send({ result })
26 // })
27
28 // parent.js
29 const worker = fork('./worker.js')
30
31 worker.send({ data: largeArray })

```

```

31 worker.on('message', ({ result }) => {
32   console.log('Worker result:', result)
33   worker.kill() // terminate when done
34 })
35
36 // execFile – run executable without shell (safer than exec!)
37 execFile('/usr/bin/python3', ['script.py', 'arg1'], (err, stdout) => {
38   console.log(stdout)
39 })
40
41 // IMPORTANT: Never exec user input directly!
42 // ■ const { stdout } = await execAsync(`ls ${userInput}`) // shell injection!
43 // ■ Use execFile or spawn with args array
44 const safe = spawn('ls', [userInput], { shell: false })

```

■ Interview Tip:

NEVER pass user input to `exec()` — it runs in a shell and enables command injection. If a user sends `; rm -rf /`, `exec('ls ' + userInput)` would delete your entire filesystem. Use `spawn()` or `execFile()` with separate args array instead.

What is cluster module? How does it work?

■ Answer

The cluster module allows you to create multiple worker processes that share the same server port. Each worker is a separate Node.js process with its own event loop.

Why cluster?

- Node.js is single-threaded — only uses 1 CPU core by default
- Cluster creates N workers — each uses 1 core
- All workers share the same port (master distributes connections)
- If one worker crashes — others keep serving

How connection distribution works:

- Linux: Round-robin (default)
- Windows: OS-level distribution
- Worker with fewest connections gets next request

Cluster vs Worker Threads:

- Cluster: separate processes, separate memory, for I/O scaling
- Worker Threads: threads in same process, can share memory, for CPU tasks

```
1  const cluster = require('cluster')
2  const os = require('os')
3  const process = require('process')
4
5  const numCPUs = os.cpus().length
6
7  if (cluster.isPrimary) { // cluster.isMaster (deprecated)
8    console.log(`Master PID ${process.pid} running`)
9    console.log(`Spawning ${numCPUs} workers...`)
10
11    // Fork workers
12    for (let i = 0; i < numCPUs; i++) {
13      const worker = cluster.fork()
14      console.log(`Worker ${worker.id} started (PID: ${worker.process.pid})`)
15    }
16
17    // Restart crashed workers
18    cluster.on('exit', (worker, code, signal) => {
19      console.log(`Worker ${worker.id} died (code:${code}, signal:${signal})`)
20      console.log('Starting replacement worker...')
21      cluster.fork() // replace dead worker
22    })
23
24    // Track worker messages (stats, logs)
25    cluster.on('message', (worker, message) => {
26      if (message.type === 'log') {
27        console.log(`[Worker ${worker.id}]:`, message.data)
28      }
29    })
30  }
```

```

30
31 // Graceful reload: restart workers one-by-one (zero downtime!)
32 process.on('SIGHUP', () => {
33   const workers = Object.values(cluster.workers)
34   let i = 0
35   function restartNext() {
36     if (i >= workers.length) return console.log('All workers reloaded!')
37     const worker = workers[i++]
38     worker.once('exit', () => {
39       cluster.fork().once('listening', restartNext)
40     })
41     worker.kill('SIGTERM')
42   }
43   restartNext()
44 })
45
46 } else {
47   // Worker process
48   const express = require('express')
49   const app = express()
50
51   app.get('/', (req, res) => {
52     res.json({ worker: process.pid, uptime: process.uptime() })
53   })
54
55   app.listen(3000, () => {
56     console.log(`Worker ${process.pid} listening on :3000`)
57     process.send({ type: 'log', data: 'ready' }) // message to master
58   })
59 }

```

■ Interview Tip:

PM2 (pm2 start app.js -i max) does all this clustering automatically with monitoring, logging, and zero-downtime reloads. Use raw cluster module only when you need custom IPC or non-standard worker management.

What are Worker Threads? How to use them?

■ Answer

Worker Threads provide true parallelism in Node.js for CPU-intensive operations. Unlike cluster (separate processes), workers share memory via SharedArrayBuffer.

When to use Worker Threads:

- Image processing (resize, convert, watermark)
- PDF generation from large templates
- Heavy mathematical computations
- Parsing and processing large datasets
- Cryptographic operations (heavy hashing)
- Any task blocking event loop > 50ms

Communication:

- `postMessage()` — send data (COPIED by default)
- Transferable objects — zero-copy transfer
- `SharedArrayBuffer` — shared memory (advanced)

Worker pool pattern:

- Pre-create N workers instead of creating per request
- Reuse workers across many requests
- Avoids overhead of thread creation

```

1  const { Worker, isMainThread, parentPort, workerData, threadId } = require('worker_threads')
2
3  if (!isMainThread) {
4    // ■■ WORKER CODE ■■
5    const { task, data } = workerData
6
7    switch(task) {
8      case 'fibonacci':
9        function fib(n) { return n < 2 ? n : fib(n-1) + fib(n-2) }
10       parentPort.postMessage({ result: fib(data), threadId })
11       break
12
13      case 'processArray':
14        const result = data.map(x => x * x).filter(x => x > 100).reduce((a, b) => a + b, 0)
15        parentPort.postMessage({ result, threadId })
16        break
17    }
18  } else {
19    // ■■ MAIN THREAD CODE ■■
20
21    // Run worker task – returns Promise
22    function runWorker(task, data) {
23      return new Promise((resolve, reject) => {
24        const worker = new Worker(__filename, { workerData: { task, data } })
25        worker.on('message', resolve)
26
27        worker.on('error', reject)

```



```

27     worker.on('exit', code => {
28         if (code !== 0) reject(new Error(`Worker exit code: ${code}`))
29     })
30 })
31 }
32
33 // Run 4 fibonacci calculations in PARALLEL
34 const [r1, r2, r3, r4] = await Promise.all([
35     runWorker('fibonacci', 40), // runs on core 1
36     runWorker('fibonacci', 40), // runs on core 2
37     runWorker('fibonacci', 40), // runs on core 3
38     runWorker('fibonacci', 40), // runs on core 4
39 ])
40 console.log('All done:', r1.result, r2.result) // truly parallel!
41
42 // SharedArrayBuffer – zero-copy shared memory
43 const sharedBuffer = new SharedArrayBuffer(4)
44 const sharedArray = new Int32Array(sharedBuffer)
45 Atomics.store(sharedArray, 0, 42) // atomic write
46
47 const w = new Worker('./compute.js', { workerData: { sharedBuffer } })
48 // Worker reads sharedArray[0] = 42 (no copying!)
49 }

```

■ Interview Tip:

Worker Threads shine for image/PDF processing in web apps. When a user requests a PDF export, spawn a worker thread to generate it. The main thread stays responsive for other requests while the worker churns through the template.

What is the EventEmitter class?

■ Answer

EventEmitter implements the Observer/Publisher-Subscriber pattern in Node.js.

Many core modules extend EventEmitter: Stream, http.Server, net.Socket, process.

Key methods:

- `emitter.on(event, fn)` — add persistent listener
- `emitter.once(event, fn)` — one-time listener (auto-removes after firing)
- `emitter.emit(event, ...args)` — trigger event with arguments
- `emitter.off(event, fn)` — remove specific listener
- `emitter.removeAllListeners(event?)` — remove all (or all for event)
- `emitter.listeners(event)` — array of listeners
- `emitter.listenerCount(event)` — number of listeners

Important behaviors:

- Default max 10 listeners per event (memory leak warning!)
- 'error' event is special — throws if no listener!
- Listeners fire synchronously in order added
- Use `setMaxListeners(n)` to increase limit

```

1  const EventEmitter = require('events')
2
3  // Create custom EventEmitter
4  class PaymentService extends EventEmitter {
5    async processPayment(order) {
6      try {
7        const payment = await chargeCard(order)
8
9        // Emit events — other services subscribe
10       this.emit('payment:success', { order, payment })
11       this.emit('payment:complete', payment.id)
12       return payment
13     } catch (err) {
14       this.emit('payment:failed', { order, error: err })
15       this.emit('error', err) // 'error' event must be handled!
16       throw err
17     }
18   }
19 }
20
21 const paymentService = new PaymentService()
22 paymentService.setMaxListeners(20) // increase from default 10
23
24 // Different services subscribe to events — loose coupling!
25 paymentService.on('payment:success', async ({ order, payment }) => {
26   await emailService.sendReceipt(order.userId, payment) // send receipt
27 })
28

```

```

29 paymentService.on('payment:success', async ({ order }) => {

```

```

30   await inventoryService.reduceStock(order.items) // reduce stock
31 })
32
33 paymentService.on('payment:failed', async ({ order, error }) => {
34   await notificationService.alertAdmin(error) // alert on failure
35 })
36
37 // MUST handle 'error' event or it throws!
38 paymentService.on('error', (err) => {
39   logger.error('PaymentService error:', err)
40 })
41
42 // once – fire exactly one time
43 paymentService.once('payment:complete', (id) => {
44   analytics.trackFirstPayment(id) // log first payment only
45 })
46
47 // List all listeners
48 console.log(paymentService.listenerCount('payment:success')) // 2
49 console.log(paymentService.listeners('payment:success')) // [fn1, fn2]

```

■ Interview Tip:

Always handle the 'error' event on EventEmitters. If an error event fires with no listener, Node.js throws an `uncaughtException` and crashes. This is a common source of production crashes in Node.js apps.

What is the net module? TCP servers?

■ Answer

The net module provides an API for creating TCP/IPC servers and clients. HTTP is built on TCP — understanding net helps you understand HTTP internals.

Use cases for raw TCP:

- Custom binary protocols
- Game servers (low latency)
- IoT device communication
- Database clients (MySQL, Redis use TCP internally)
- IPC (Inter-Process Communication) via Unix domain sockets

net.Socket:

- Duplex stream — readable AND writable
- Events: 'connect', 'data', 'end', 'close', 'error'
- Methods: socket.write(), socket.end(), socket.destroy()

```
1  const net = require('net')
2
3  // TCP Server
4  const server = net.createServer((socket) => {
5    console.log(`Client connected: ${socket.remoteAddress}:${socket.remotePort}`)
6
7    socket.on('data', (data) => {
8      const message = data.toString().trim()
9      console.log(`Received: ${message}`)
10
11      // Echo back with modification
12      socket.write(`Echo: ${message}\n`)
13
14      if (message === 'quit') {
15        socket.end() // graceful close
16      }
17    })
18
19    socket.on('end', () => console.log('Client disconnected'))
20    socket.on('error', (err) => console.error('Socket error:', err.message))
21  })
22
23  server.listen(8080, '0.0.0.0', () => {
24    console.log('TCP server on port 8080')
25    // Test: nc localhost 8080
26  })
27
28  // TCP Client
29  const client = net.createConnection({ port: 8080, host: 'localhost' }, () => {
30    console.log('Connected to server!')
31    client.write('Hello Server!\n')
```

```
32  })
```

```
33
34 client.on('data', (data) => console.log('Server says:', data.toString()))
35 client.on('end', () => console.log('Disconnected from server'))
36
37 // IPC via Unix Domain Socket (faster than TCP for local)
38 const IPC_PATH = '/tmp/my-app.sock'
39 const ipcServer = net.createServer((socket) => {
40   socket.on('data', (data) => {
41     const msg = JSON.parse(data.toString())
42     socket.write(JSON.stringify({ reply: 'received', id: msg.id }))
43   })
44 })
45 ipcServer.listen(IPC_PATH)
```

■ Interview Tip:

Unix domain sockets (/tmp/my-app.sock) are 2-3x faster than TCP localhost connections because they skip the TCP/IP stack entirely. Redis and PostgreSQL support Unix socket connections for this reason.

Q23 What is graceful shutdown? How to implement it?

■ Answer

Graceful shutdown means stopping a server in a controlled way: stop new connections, finish in-flight requests, close DB connections, then exit.

Why graceful shutdown matters:

- Kubernetes sends SIGTERM before killing a pod (default 30s grace period)
- Without it: in-flight requests get killed → clients see errors
- Database connections not closed → connection pool exhausted
- Kafka/Redis consumers not closed → messages lost

Shutdown sequence:

1. Stop accepting new connections (`server.close()`)
2. Wait for active requests to complete
3. Close database connections
4. Close cache connections (Redis)
5. Flush logs
6. Exit with code 0

Shutdown timeout:

- Set a max timeout (30s) — force exit if something hangs
- `process.exit(1)` after timeout to prevent stuck containers

```
1  const express = require('express')
2  const mongoose = require('mongoose')
3  const redis = require('ioredis')
4
5  const app = express()
6  const redisClient = new redis(process.env.REDIS_URL)
7  const server = app.listen(process.env.PORT || 3000)
8
9  // Track active connections
10 let activeConnections = 0
11 app.use((req, res, next) => {
12   activeConnections++
13   res.on('finish', () => activeConnections--)
14   res.on('close', () => activeConnections--)
15   next()
16 })
17
18 let isShuttingDown = false
19 app.use((req, res, next) => {
20   if (isShuttingDown) {
21     res.setHeader('Connection', 'close') // tell client to reconnect elsewhere
22     return res.status(503).json({ error: 'Server shutting down, please retry' })
23   }
24   next()
25 })
```

```
26
```

```

27 async function gracefulShutdown(signal) {
28   console.log(`\n${signal} received – starting graceful shutdown...\`)
29   isShuttingDown = true
30
31   // Force shutdown after 30 seconds
32   const forceExit = setTimeout(() => {
33     console.error('Forced shutdown after 30s timeout')
34     process.exit(1)
35   }, 30000).unref() // unref = won't prevent exit
36
37   try {
38     // 1. Stop accepting new connections
39     await new Promise((resolve) => server.close(resolve))
40     console.log('HTTP server closed')
41
42     // 2. Close database connection
43     await mongoose.connection.close()
44     console.log('MongoDB disconnected')
45
46     // 3. Close Redis
47     await redisClient.quit()
48     console.log('Redis disconnected')
49
50     clearTimeout(forceExit)
51     console.log('Graceful shutdown complete')
52     process.exit(0)
53   } catch (err) {
54     console.error('Shutdown error:', err)
55     process.exit(1)
56   }
57 }
58
59 process.on('SIGTERM', () => gracefulShutdown('SIGTERM')) // K8s, Docker stop
60 process.on('SIGINT', () => gracefulShutdown('SIGINT')) // Ctrl+C

```

■ Interview Tip:

In Kubernetes, set `terminationGracePeriodSeconds` to match your app's max shutdown time. K8s waits this long after SIGTERM before sending SIGKILL. If your app takes 15 seconds to drain, set `terminationGracePeriodSeconds: 30`.

Q24 What is error handling in Node.js? Best practices?

■ Answer

Proper error handling prevents crashes and provides good user experience. Node.js has several types of errors that need different handling strategies.

Types of errors:

- Operational errors — expected: network failure, invalid input, DB connection refused
- Programmer errors — bugs: TypeError, undefined property access

Error handling mechanisms:

- try/catch — synchronous and async/await errors
- Promise .catch() — promise rejections
- EventEmitter 'error' event — stream and socket errors
- process.uncaughtException — last resort for sync errors
- process.unhandledRejection — last resort for promise rejections

Error-first callback convention:

- First argument: error (or null if success)
- Second argument: result

Custom Error classes:

- Extend Error for meaningful error types
- Add statusCode, code, isOperational properties

```
1 // Custom error hierarchy
2 class AppError extends Error {
3   constructor(message, statusCode = 500, code = 'INTERNAL_ERROR') {
4     super(message)
5     this.name = this.constructor.name
6     this.statusCode = statusCode
7     this.code = code
8     this.isOperational = true // we know about this error
9     Error.captureStackTrace(this, this.constructor)
10  }
11 }
12
13 class NotFoundError extends AppError {
14   constructor(resource) { super(`${resource} not found`, 404, 'NOT_FOUND') }
15 }
16 class ValidationError extends AppError {
17   constructor(msg, details) {
18     super(msg, 400, 'VALIDATION_ERROR')
19     this.details = details
20   }
21 }
22 class UnauthorizedError extends AppError {
23   constructor(msg = 'Unauthorized') { super(msg, 401, 'UNAUTHORIZED') }
24 }
```



```

26 // Error-first callback pattern
27 fs.readFile(path, (err, data) => {
28   if (err) {
29     if (err.code === 'ENOENT') throw new NotFoundError('config file')
30     throw err // unknown error, re-throw
31   }
32   process(data)
33 })
34
35 // Async/await error handling
36 async function getData(id) {
37   try {
38     const data = await db.find(id)
39     if (!data) throw new NotFoundError('Record')
40     return data
41   } catch (err) {
42     if (err instanceof AppError) throw err // re-throw operational
43     // Wrap programmer errors
44     throw new AppError(`Unexpected error: ${err.message}`)
45   }
46 }
47
48 // Global handlers (last resort – log and restart!)
49 process.on('uncaughtException', (err) => {
50   logger.fatal({ err }, 'UNCAUGHT EXCEPTION – process will exit')
51   process.exit(1) // PM2/Docker will restart
52 })
53 process.on('unhandledRejection', (reason, promise) => {
54   logger.error({ reason, promise }, 'UNHANDLED REJECTION')
55   process.exit(1)
56 })

```

■ Interview Tip:

Never ignore unhandled promise rejections — in Node.js 15+, they crash the process. Always add `.catch()` to standalone promise chains or use `async/await` with `try/catch`. The `process.on('unhandledRejection')` handler should log and exit — PM2 will restart.

■ Answer

Promises represent eventual completion (or failure) of an asynchronous operation. They solve callback hell by enabling chaining and better error handling.

Promise states:

- pending → initial state
- fulfilled → resolve() was called
- rejected → reject() was called
- settled = fulfilled OR rejected (final, immutable)

Promise methods:

- .then(onFulfill, onReject) — handle resolve/reject
- .catch(onReject) — handle errors (shorthand for .then(null, fn))
- .finally(fn) — always runs regardless of outcome

Promise static methods:

- Promise.all([p1,p2]) — all succeed or first failure
- Promise.allSettled([p1,p2]) — all complete, get all results
- Promise.race([p1,p2]) — first to settle (succeed or fail)
- Promise.any([p1,p2]) — first to succeed
- Promise.resolve(val) — create resolved promise
- Promise.reject(err) — create rejected promise

```
1 // Creating promises
2 const delay = (ms) => new Promise(resolve => setTimeout(resolve, ms))
3
4 const fetchUser = (id) => new Promise((resolve, reject) => {
5   setTimeout(() => {
6     if (id > 0) resolve({ id, name: 'Kaushal' })
7     else reject(new Error('Invalid ID'))
8   }, 100)
9 })
10
11 // Promise chaining
12 fetchUser(1)
13   .then(user => fetchPosts(user.id)) // return value passed to next .then
14   .then(posts => { console.log(posts); return posts.length })
15   .then(count => console.log(`Total posts: ${count}`))
16   .catch(err => console.error('Error:', err.message))
17   .finally(() => setLoading(false)) // always runs
18
19 // Promise.all — parallel, all must succeed
20 try {
21   const [user, posts, settings] = await Promise.all([
22     fetchUser(1),
23     fetchPosts(1),
24     fetchSettings(1)
```

```
25   ]) // if ANY fails, all fail
```

```

26 } catch(err) { /* one of them failed */ }
27
28 // Promise.allSettled – parallel, get all results
29 const results = await Promise.allSettled([p1, p2, p3])
30 results.forEach(result => {
31   if (result.status === 'fulfilled') console.log('OK:', result.value)
32   else console.log('Failed:', result.reason.message)
33 })
34
35 // Promise.race – timeout pattern
36 const timeout = (ms) => new Promise( (_, reject) =>
37   setTimeout(() => reject(new Error(`Timeout after ${ms}ms`)), ms))
38
39 const result = await Promise.race([
40   fetchData(),           // actual work
41   timeout(5000)           // 5 second timeout
42 ])
43
44 // Promise.any – first to SUCCEED (opposite of race)
45 const fastest = await Promise.any([
46   fetchFromServer1(),
47   fetchFromServer2(),
48   fetchFromServer3()
49 ]) // returns first successful result

```

■ Interview Tip:

Promise.race with a timeout is a production pattern for API calls. Never let your app wait forever for an external service. Set a timeout: if the API doesn't respond in 5 seconds, fail fast and return an error or cached data.

■ Answer

async/await is syntactic sugar over Promises. It makes async code look synchronous. An async function always returns a Promise — even if you return a non-Promise value.

Key rules:

- async marks a function as asynchronous
- await pauses execution until Promise resolves
- await can only be used inside async functions
- await does NOT block the thread — event loop continues!
- try/catch handles both sync and async errors

Common patterns:

- Sequential — await one after another
- Parallel — await Promise.all([...])
- Loop — for...of with await (NOT forEach!)
- Error handling — try/catch or .catch() on result

Performance pitfall:

- await in loop = sequential (slow!)
- Use Promise.all for parallel execution

```
1 // async/await basics
2 async function getUser(id) {
3   const user = await db.users.findById(id) // pauses here, not the thread
4   if (!user) throw new NotFoundError('User') // throw works normally
5   return user // auto-wrapped in Promise.resolve(user)
6 }
7
8 // ■ SLOW: sequential (waits for each one)
9 async function loadDataSlowly() {
10  const user = await fetchUser(1) // 500ms
11  const posts = await fetchPosts(1) // 300ms
12  const stats = await fetchStats(1) // 200ms
13  // Total: 1000ms (500+300+200)
14 }
15
16 // ■ FAST: parallel (all run simultaneously)
17 async function loadDataFast() {
18  const [user, posts, stats] = await Promise.all([
19    fetchUser(1), // all start at same time
20    fetchPosts(1), // all run simultaneously
21    fetchStats(1)
22  ])
23  // Total: 500ms (max of all three)
24 }
25
26 // ■ await in forEach — doesn't work!
```

```
27 async function wrongLoop() {
```

```

28  const ids = [1, 2, 3, 4, 5]
29  ids.forEach(async id => {
30    await processItem(id) // forEach doesn't wait for async!
31  })
32  console.log('Done?') // NOT done! All running in background
33  }
34
35  // ■ for...of loop – awaits properly
36  async function correctSequentialLoop() {
37    const ids = [1, 2, 3, 4, 5]
38    for (const id of ids) {
39      await processItem(id) // waits before next iteration
40    }
41    console.log('Actually done!') // guaranteed
42  }
43
44  // ■ Process all in parallel with for...of
45  async function parallelLoop() {
46    const ids = [1, 2, 3, 4, 5]
47    await Promise.all(ids.map(id => processItem(id))) // all at once
48  }
49
50  // Error handling patterns
51  async function withError() {
52    // Pattern 1: try/catch
53    try { return await riskyOperation() }
54    catch(err) { logger.error(err); return null }
55
56    // Pattern 2: .catch() on awaited result
57    const data = await riskyOperation().catch(() => null)
58  }

```

■ Interview Tip:

The forEach + await bug is the #1 most common async mistake. forEach calls the callback function but ignores its return value (the Promise). Use for...of for sequential, Promise.all(array.map(fn)) for parallel.

What is the event-driven architecture?

■ Answer

Event-driven architecture decouples components by having them communicate through events. Producers emit events; consumers react to events; neither knows about the other.

Benefits:

- Loose coupling — components don't depend on each other directly
- Easy to add new consumers without changing producers
- Natural fit for Node.js's event-based model
- Enables async processing and better scalability

In Node.js:

- EventEmitter — synchronous, in-process
- Message queues (BullMQ, Kafka) — async, across services

Patterns:

- Observer — object notifies dependents of changes
- Pub/Sub — publishers emit to topics; subscribers receive from topics
- Event Sourcing — store events as source of truth

```
1  const EventEmitter = require('events')
2
3  // Domain-driven event system
4  class DomainEventBus extends EventEmitter {
5    publish(event, data) {
6      const envelope = {
7        eventType: event,
8        timestamp: new Date().toISOString(),
9        correlationId: crypto.randomUUID(),
10       data
11     }
12     this.emit(event, envelope)
13     this.emit('*', envelope) // wildcard - log everything
14   }
15 }
16
17 const bus = new DomainEventBus()
18
19 // USER SERVICE — publishes user events
20 class UserService {
21   async register(userData) {
22     const user = await User.create(userData)
23     bus.publish('user.registered', { userId: user._id, email: user.email })
24     return user
25   }
26   async login(credentials) {
27     const user = await authenticate(credentials)
28     bus.publish('user.loggedIn', { userId: user._id, ip: credentials.ip })
29     return user
30   }
31 }
```

```

30 }
31 }
32
33 // EMAIL SERVICE – reacts to user events
34 bus.on('user.registered', async ({ data }) => {
35   await emailService.sendWelcome(data.email)
36 })
37
38 // ANALYTICS SERVICE – reacts to all events
39 bus.on('*', ({ eventType, timestamp, data }) => {
40   analyticsDB.insert({ eventType, timestamp, data })
41 })
42
43 // AUDIT SERVICE – security events
44 bus.on('user.loggedIn', ({ data }) => {
45   auditLog.write(`Login: ${data.userId} from ${data.ip}`)
46 })
47
48 // Adding new feature? Just add a new listener!
49 bus.on('user.registered', ({ data }) => {
50   recommenderService.initializeProfile(data.userId)
51 })

```

■ Interview Tip:

Event-driven architecture is how microservices communicate without tight coupling. Adding a new feature (like recommendations on user registration) just means adding a new event listener — no changes to UserService needed.

What is AsyncLocalStorage?

■ Answer

AsyncLocalStorage provides async-local storage — context that propagates through async call chains. Like thread-local storage in multi-threaded systems, but for async contexts.

Use cases:

- Request context (requestId, userId, traceId) throughout call chain
- Per-request database transactions
- Distributed tracing (correlation IDs)
- Structured logging with automatic context

How it works:

- `storage.run(value, callback)` — set value for this async context
- `storage.getStore()` — get current value from anywhere in async chain
- Value propagates through: `await`, `callbacks`, `setTimeout`, etc.
- Each async context has its own isolated storage
- Available since Node.js 12.17.0

```
1  const { AsyncLocalStorage } = require('async_hooks')
2  const crypto = require('crypto')
3
4  // Create storage for request context
5  const requestContext = new AsyncLocalStorage()
6
7  // Logger that automatically includes request context
8  const logger = {
9    log(level, message, extra = {}) {
10      const ctx = requestContext.getStore() || {}
11      console.log(JSON.stringify({
12        level, message,
13        requestId: ctx.requestId, // automatic! no need to pass
14        userId: ctx.userId,
15        timestamp: new Date().toISOString(),
16        ...extra
17      }))
18    },
19    info: (msg, data) => logger.log('INFO', msg, data),
20    error: (msg, data) => logger.log('ERROR', msg, data)
21  }
22
23  // Express middleware – set context for each request
24  app.use((req, res, next) => {
25    const ctx = {
26      requestId: req.headers['x-request-id'] || crypto.randomUUID(),
27      userId: req.user?.id || null,
28      method: req.method,
29      url: req.url
30    }
31  })
```

```
31  // All code called from next() can access this context!
```



```

32   requestContext.run(ctx, next)
33 })
34
35 // Anywhere in the call chain – no prop drilling!
36 async function processOrder(orderId) {
37   logger.info('Processing order', { orderId }) // requestId auto-included
38   const order = await Order.findById(orderId)
39   logger.info('Order fetched', { status: order.status })
40   await payment.charge(order)
41   logger.info('Payment processed') // all with same requestId!
42   return order
43 }
44
45 // Route handler – no ctx parameter needed anywhere!
46 app.post('/orders/:id/process', async (req, res) => {
47   const order = await processOrder(req.params.id) // no ctx passing
48   res.json(order)
49 })

```

■ Interview Tip:

AsyncLocalStorage is the modern solution to 'I need the requestId in my database query helper but don't want to thread it through every function call.' Set it once in middleware, read it anywhere in the call chain.

What is require() cache in Node.js?

■ Answer

Node.js caches modules after first `require()` — subsequent `require()` calls return the cached version. This means module code runs ONCE, and the same exports object is returned every time.

How module cache works:

- First `require('./module')` — loads, executes, caches
- Second `require('./module')` — returns from cache (no re-execution!)
- Cache key = resolved absolute file path
- Modules are singletons by default

Practical implications:

- State in a module is shared (database connections, configs)
- Circular dependencies are handled (partially)
- Hot-reloading requires cache busting

Accessing the cache:

- `require.cache` — object with all cached modules
- `delete require.cache[require.resolve('./module')]` — bust cache

```
1 // Module caching demonstration
2
3 // counter.js
4 let count = 0
5 module.exports = {
6   increment: () => ++count,
7   getCount: () => count
8 }
9
10 // main.js – shows singleton behavior
11 const counter1 = require('./counter')
12 const counter2 = require('./counter') // same object! (cached)
13
14 counter1.increment() // count = 1
15 counter2.increment() // count = 2 (same module!)
16 console.log(counter1.getCount()) // 2 – they share state
17 console.log(counter1 === counter2) // true – literally same object
18
19 // Database connection singleton pattern (relies on cache!)
20 // db.js
21 const mongoose = require('mongoose')
22 let connection = null
23
24 async function connect() {
25   if (connection) return connection // already connected
26   connection = await mongoose.connect(process.env.MONGO_URI)
27   return connection
28 }
```

```
29 module.exports = { connect, getConnection: () => connection }
```



```
30
31 // app.js + routes.js – both require same module = same connection!
32 require('./db').connect() // connects once
33 require('./routes')       // routes also require('./db') – gets same connection
34
35 // Cache busting (for hot reload in development)
36 function reloadModule(modulePath) {
37   const resolved = require.resolve(modulePath)
38   delete require.cache[resolved] // remove from cache
39   return require(modulePath)     // fresh load
40 }
41
42 // View all cached modules
43 Object.keys(require.cache).forEach(key => {
44   console.log(key) // absolute paths of all loaded modules
45 })
```

■ Interview Tip:

The module cache is why database connection modules work as singletons. `require('./db')` in 100 different files all return the same connection pool. This is intentional — you don't want 100 separate connection pools!

What is top-level await in Node.js?

■ Answer

Top-level await allows using await outside of async functions in ES Modules. Available in Node.js 14.8+ with ES Modules (import/export).

What it enables:

- Await module-level async operations during module loading
- Dynamic imports at top level
- Conditional module loading
- Initialization that requires async operations

Requires:

- ES Modules — file must be .mjs OR package.json type:module
- Node.js 14.8+

NOT available in:

- CommonJS (require/module.exports)
- Regular .js files without type:module

When to use:

- Database connection at module level
- Loading configuration from async sources
- Conditional module selection

```
1 // Only works in .mjs files or type:module packages!
2
3 // config.mjs – async configuration loading
4 import { readFile } from 'fs/promises'
5
6 // Top-level await! No async function wrapper needed
7 const configData = await readFile('./config.json', 'utf8')
8 export const config = JSON.parse(configData)
9
10 // db.mjs – async connection at module level
11 import mongoose from 'mongoose'
12
13 // Connect before module exports are available
14 await mongoose.connect(process.env.MONGODB_URI)
15 console.log('Database connected!')
16
17 export const db = mongoose.connection
18
19 // app.mjs – importing modules that use top-level await
20 import { config } from './config.mjs' // waits for config to load
21 import { db } from './db.mjs'        // waits for DB to connect
22
23 // By the time these run, config and db are fully initialized
24 console.log('App starting with config:', config.appName)
```

```
26 // Dynamic import with top-level await
27 const moduleName = process.env.FEATURE || 'default'
28 const { handler } = await import(`./handlers/${moduleName}.mjs`)
29
30 // Conditional loading based on environment
31 let adapter
32 if (process.env.NODE_ENV === 'test') {
33   adapter = await import('./adapters/memory.mjs')
34 } else {
35   adapter = await import('./adapters/redis.mjs')
36 }
37
38 // ■ This doesn't work in CommonJS (.js files):
39 // const data = await fetch('/api') // SyntaxError: await outside async
```

■ Interview Tip:

Top-level await is a game-changer for module initialization. Instead of `init()` patterns or wrapping everything in `async`, you can directly await at module level. Just remember: it only works with ES Modules, not CommonJS.

What is the DNS module in Node.js?

■ Answer

The `dns` module provides DNS (Domain Name System) resolution functions. Two different approaches with different performance characteristics.

`dns.lookup()` vs `dns.resolve()`:

- `dns.lookup()` — uses OS resolver (`getaddrinfo`) — goes through libuv thread pool
- `dns.resolve()` — direct DNS server query — non-blocking event loop I/O

Why this matters:

- `dns.lookup()` blocks a libuv thread (from thread pool of 4)
- Hundreds of concurrent `dns.lookup()` can exhaust thread pool
- `dns.resolve()` is truly async — no thread pool needed
- For production: use `dns.resolve()` for bulk lookups

Common operations:

- `resolve4/resolve6` — IPv4/IPv6 addresses
- `resolveMx` — mail server records
- `resolveTxt` — text records
- `resolveNs` — name server records
- `reverse` — IP to hostname

```
1  const dns = require('dns')
2  const { resolve4, resolve6, resolveMx, lookup } = require('dns/promises')
3
4  // lookup — uses OS, checks /etc/hosts and local cache
5  const { address, family } = await lookup('google.com')
6  console.log(`google.com → ${address} (IPv${family})`)
7
8  // resolve4 — query DNS server directly (recommended for production)
9  const addresses = await resolve4('google.com')
10 console.log('IPv4 addresses:', addresses) // ['142.250.194.46', ...]
11
12 // Multiple IPs = round-robin load balancing by DNS
13 const apiIps = await resolve4('api.example.com')
14 console.log(`API has ${apiIps.length} servers:`, apiIps)
15
16 // Email validation — check MX records
17 async function validateEmailDomain(email) {
18   const domain = email.split('@')[1]
19   try {
20     const mxRecords = await resolveMx(domain)
21     return mxRecords.length > 0 // domain has mail servers
22   } catch {
23     return false // domain doesn't exist or no MX records
24   }
25 }
26
```

```
27 // Reverse DNS — get hostname from IP
```

```
28 dns.reverse('8.8.8.8', (err, hostnames) => {
29   console.log(hostnames) // ['dns.google']
30 })
31
32 // Set custom DNS server
33 dns.setServers(['1.1.1.1', '8.8.8.8']) // Cloudflare + Google DNS
34
35 // Check if domain exists (for URL validation)
36 async function domainExists(domain) {
37   try { await resolve4(domain); return true }
38   catch { return false }
39 }
```

■ Interview Tip:

In production code that makes many DNS lookups (like a link checker), use `dns.resolve()` not `dns.lookup()`. With 4 libuv threads and hundreds of concurrent lookups, `dns.lookup()` will cause serious thread pool starvation.

What is the cluster module's IPC?

■ Answer

IPC (Inter-Process Communication) allows master and worker processes to exchange messages. Workers and master communicate via `process.send()` and `'message'` event.

Use cases for cluster IPC:

- Aggregating statistics from all workers
- Distributing shared configuration updates
- Coordinating rate limiting across workers
- Health monitoring from master
- Hot config reload signals

Limitations:

- Messages are serialized (JSON) — no shared memory
- Only between master and workers — not worker to worker directly
- For worker-to-worker: route through master

Alternative for shared state:

- Redis — shared cache accessible to all workers
- SharedArrayBuffer (Worker Threads only)

```
1  const cluster = require('cluster')
2  const os = require('os')
3
4  if (cluster.isPrimary) {
5      const stats = { requests: 0, errors: 0, workers: {} }
6
7      for (let i = 0; i < os.cpus().length; i++) {
8          const worker = cluster.fork()
9
10         // Receive messages from workers
11         worker.on('message', (msg) => {
12             if (msg.type === 'STATS') {
13                 stats.workers[worker.id] = msg.data
14                 stats.requests += msg.data.requests
15                 stats.errors += msg.data.errors
16             }
17             if (msg.type === 'READY') {
18                 console.log(`Worker ${worker.id} is ready`)
19             }
20         })
21     }
22
23     // Broadcast config update to all workers
24     function broadcastConfig(config) {
25         for (const id in cluster.workers) {
26             cluster.workers[id].send({ type: 'CONFIG_UPDATE', data: config })
27         }
28     }
```



```

29
30 // Collect stats every 30 seconds
31 setInterval(() => {
32     for (const id in cluster.workers) {
33         cluster.workers[id].send({ type: 'GET_STATS' })
34     }
35     console.log('Cluster stats:', stats)
36 }, 30000)
37
38 } else {
39     const workerStats = { requests: 0, errors: 0 }
40     let config = {}
41
42     // Listen to messages from master
43     process.on('message', (msg) => {
44         if (msg.type === 'CONFIG_UPDATE') {
45             config = msg.data
46             console.log(`Worker ${process.pid} config updated`)
47         }
48         if (msg.type === 'GET_STATS') {
49             process.send({ type: 'STATS', data: workerStats })
50         }
51     })
52
53     const app = require('./app')(config)
54     app.listen(3000, () => {
55         process.send({ type: 'READY' }) // tell master we're ready
56     })
57 }

```

■ Interview Tip:

Cluster IPC is useful but complex. For simple shared state (rate limits, feature flags), use Redis instead — it's simpler and works across multiple machines too. Use cluster IPC for coordination that's specific to a single machine's worker processes.

How to handle memory leaks in Node.js?

■ Answer

Memory leaks cause the process to use more and more memory over time until it crashes. In long-running servers, even small leaks become critical over hours/days.

Common memory leak sources:

- Event listeners not removed — most common!
- Closures holding large objects
- Global variables accumulating data
- Timers (setInterval) not cleared
- Caches without size limits or eviction
- Module-level arrays that grow indefinitely

Detection tools:

- Chrome DevTools heap snapshots
- clinic.js (clinic doctor, clinic heapprofiler)
- --expose-gc flag + global.gc()
- node --inspect for live monitoring
- process.memoryUsage() over time

Prevention strategies:

- Always remove event listeners when done
- Use WeakMap/WeakSet for object-keyed data
- Implement LRU caches with size limits
- Clear intervals/timeouts when no longer needed

```
1  const EventEmitter = require('events')
2
3  // ■ LEAK 1: Event listener not removed
4  class BadService extends EventEmitter {
5    start() {
6      process.on('SIGTERM', this.shutdown.bind(this)) // new listener each start!
7    }
8  }
9
10 // ■ FIX: Track and remove listener
11 class GoodService extends EventEmitter {
12   constructor() { super(); this._shutdownHandler = this.shutdown.bind(this) }
13   start() { process.on('SIGTERM', this._shutdownHandler) }
14   stop() { process.off('SIGTERM', this._shutdownHandler) } // cleanup!
15 }
16
17 // ■ LEAK 2: Growing cache without eviction
18 const cache = {} // grows indefinitely!
19 app.use((req, res, next) => {
20   cache[req.url] = { response: 'large data', time: Date.now() }
21   next()
22 })
```



```
23
24 // ■ FIX: LRU cache with size limit
25 const LRU = require('lru-cache')
26 const safeCache = new LRU({ max: 1000, ttl: 1000 * 60 * 5 }) // max 1000 items
27
28 // ■ LEAK 3: Uncleaned setInterval
29 function startPolling() {
30   setInterval(pollDatabase, 5000) // never cleared!
31 }
32
33 // ■ FIX: Store reference and clear on shutdown
34 let pollInterval
35 function startPolling() { pollInterval = setInterval(pollDatabase, 5000) }
36 function stopPolling() { clearInterval(pollInterval) }
37
38 // Memory monitoring with auto-alert
39 setInterval(() => {
40   const { heapUsed, heapTotal } = process.memoryUsage()
41   const heapPercent = (heapUsed / heapTotal * 100).toFixed(1)
42   if (heapUsed > 500 * 1024 * 1024) { // > 500MB
43     logger.warn({ heapUsed, heapPercent }, 'High memory usage!')
44   }
45 }, 60000)
```

■ Interview Tip:

The `MaxListenersExceededWarning` is Node.js telling you about a potential listener leak. By default, 11+ listeners on same event triggers this warning. It's often a sign that you're adding listeners in a loop without removing them.

Q34 What is the diagnostic_channel and perf_hooks?

■ Answer

Node.js has built-in diagnostic APIs for performance monitoring and tracing.

perf_hooks module:

- performance.now() — high-resolution timer
- performance.mark() — mark a point in time
- performance.measure() — measure between marks
- PerformanceObserver — observe performance entries
- monitorEventLoopDelay() — measure event loop lag

diagnostics_channel module:

- Create named channels for publishing/subscribing to diagnostic events
- Used by Node.js internals and libraries (http, fs, net)
- Subscribe to channels to trace library operations

When to use:

- Profile specific operations
- Detect event loop lag
- Trace HTTP requests, database queries
- Custom performance metrics

```

1  const { performance, PerformanceObserver, monitorEventLoopDelay } = require('perf_hooks')
2  const dc = require('diagnostics_channel')
3
4  // Basic timing
5  const t0 = performance.now()
6  await heavyOperation()
7  const elapsed = performance.now() - t0
8  console.log(`Took ${elapsed.toFixed(2)}ms`)
9
10 // Named marks and measures
11 performance.mark('db-start')
12 const users = await db.users.findMany()
13 performance.mark('db-end')
14 performance.measure('db-query', 'db-start', 'db-end')
15
16 // PerformanceObserver – collect all measurements
17 const obs = new PerformanceObserver((list) => {
18   for (const entry of list.getEntries()) {
19     if (entry.duration > 100) { // alert on slow ops
20       logger.warn(`SLOW: ${entry.name} took ${entry.duration.toFixed(0)}ms`)
21     }
22     metrics.histogram(entry.name, entry.duration) // send to monitoring
23   }
24 })
25 obs.observe({ entryTypes: ['measure', 'http', 'gc'], buffered: true })
26

```

```

27 // Event loop delay monitoring – detect if loop is blocked

```

```
28 const histogram = monitorEventLoopDelay({ resolution: 20 })
29 histogram.enable()
30 setInterval(() => {
31   const lag = histogram.mean / 1e6 // nanoseconds to milliseconds
32   if (lag > 50) logger.warn(`Event loop lag: ${lag.toFixed(1)}ms`) // >50ms is bad
33   histogram.reset()
34 }, 5000)
35
36 // diagnostics_channel - subscribe to http module events
37 const httpChannel = dc.channel('http.client.request.start')
38 httpChannel.subscribe((message) => {
39   console.log('HTTP request starting:', message.request.url)
40 })
```

■ Interview Tip:

Event loop lag is one of the most important Node.js metrics in production. If your event loop lag exceeds 100ms, users are experiencing latency spikes. Set up monitoring with `monitorEventLoopDelay` and alert your team when it's consistently high.

How to scale Node.js applications?

■ Answer

Node.js scaling strategies range from simple (cluster) to complex (microservices).

Scaling approaches in order of complexity:

- Vertical scaling — bigger machine (simple, limited)
- Cluster module — use all CPU cores on one machine
- PM2 cluster mode — production-grade, zero-downtime
- Horizontal scaling — multiple machines behind load balancer
- Microservices — independent services, each scaled independently

Stateless design is required for horizontal scaling:

- No sessions in memory — use Redis
- No files on local disk — use S3/blob storage
- No in-process caches for shared data — use Redis

Load balancing strategies:

- Round Robin — even distribution
- Least Connections — route to least busy
- IP Hash — same client always goes to same server (sticky sessions)
- Resource-based — route based on CPU/memory

```
1 // Strategy 1: Cluster (single machine, all cores)
2 // pm2 start app.js -i max ← simplest production approach
3
4 // Strategy 2: Stateless Express app (required for horizontal scaling)
5 const session = require('express-session')
6 const RedisStore = require('connect-redis')(session)
7 const redis = require('ioredis').createClient()
8
9 // ■ In-memory sessions – breaks on multiple servers
10 // app.use(session({ secret: 'key', store: null }))
11
12 // ■ Redis sessions – shared across all servers
13 app.use(session({
14   store: new RedisStore({ client: redis }),
15   secret: process.env.SESSION_SECRET,
16   resave: false, saveUninitialized: false
17 }))
18
19 // Strategy 3: Health check endpoint (required for load balancers)
20 let isHealthy = true
21 app.get('/health', (req, res) => {
22   if (!isHealthy) return res.status(503).json({ status: 'unhealthy' })
23   res.json({ status: 'ok', version: process.env.npm_package_version })
24 })
25
26 // Strategy 4: Nginx load balancer config (upstream servers)
```

```
27 // upstream nodejs {
```

```
28 // least_conn; # least connections strategy
29 // server 10.0.0.1:3000;
30 // server 10.0.0.2:3000;
31 // server 10.0.0.3:3000;
32 // keepalive 64;
33 // }
34
35 // Strategy 5: Rate limiting per IP (works across all servers with Redis)
36 const rateLimit = require('express-rate-limit')
37 const RedisRateLimitStore = require('rate-limit-redis')
38 app.use(rateLimit({
39   windowMs: 15 * 60 * 1000, max: 100,
40   store: new RedisRateLimitStore({ client: redis }) // shared across servers!
41 })))
```

■ Interview Tip:

The #1 thing needed before horizontal scaling: make your app stateless. Any state that needs to be shared (sessions, rate limit counters, distributed locks) must go in Redis. Test by running two instances locally and verifying they work identically.

What is connection pooling? How to implement it?

■ Answer

Connection pooling maintains a pool of pre-created connections to reuse, avoiding the overhead of creating new connections per request.

Why pooling matters:

- Creating DB connection: ~100-300ms per connection
- Pool keeps N connections open and reuses them
- Request gets a connection from pool (< 1ms)
- Returns connection when done

Pool configuration:

- min — minimum connections to maintain always
- max — maximum connections allowed
- acquireTimeoutMillis — wait timeout for available connection
- idleTimeoutMillis — close idle connections after N ms

Too few vs too many connections:

- Too few: requests queue up waiting for connections
- Too many: overwhelm database server
- Rule of thumb: 5-10 connections per CPU core

```
1 // PostgreSQL with pg (built-in pool)
2 const { Pool } = require('pg')
3
4 const pgPool = new Pool({
5   host: process.env.PGHOST,
6   database: process.env.PGDATABASE,
7   user: process.env.PGUSER,
8   password: process.env.PGPASSWORD,
9   port: 5432,
10  max: 20,                // max connections in pool
11  min: 5,                 // keep 5 connections warm
12  idleTimeoutMillis: 30000, // close idle after 30s
13  connectionTimeoutMillis: 2000, // fail if no connection available in 2s
14  maxUses: 7500           // recycle connection after 7500 uses
15 })
16
17 // Log pool events
18 pgPool.on('connect', (client) => {
19   console.log('New client connected to pool')
20 })
21 pgPool.on('error', (err, client) => {
22   console.error('Pool error:', err.message)
23 })
24
25 // Use pool.query – auto-acquires and releases
26 const result = await pgPool.query('SELECT * FROM users WHERE id = $1', [userId])
```



```

28 // For transactions – manually acquire client
29 const client = await pgPool.connect()
30 try {
31   await client.query('BEGIN')
32   await client.query('UPDATE accounts SET balance = balance - $1 WHERE id = $2', [100, fromId])
33   await client.query('UPDATE accounts SET balance = balance + $1 WHERE id = $2', [100, toId])
34   await client.query('COMMIT')
35 } catch(err) {
36   await client.query('ROLLBACK')
37   throw err
38 } finally {
39   client.release() // MUST release back to pool!
40 }
41
42 // Mongoose (MongoDB) – connection pool
43 mongoose.connect(process.env.MONGODB_URI, {
44   maxPoolSize: 10,           // max connections
45   minPoolSize: 2,           // minimum connections
46   maxIdleTimeMS: 30000,     // close idle after 30s
47   connectTimeoutMS: 5000    // connection attempt timeout
48 })

```

■ Interview Tip:

Always call `client.release()` in a finally block. If an exception occurs and `release()` isn't called, the connection is leaked from the pool. After enough leaks, all pool connections are in use and new requests timeout.

What is Node.js performance optimization?

■ Answer

Performance optimization in Node.js requires identifying bottlenecks first — don't optimize blindly.

Profiling tools:

- clinic.js doctor — identifies event loop issues
- clinic.js flame — flame graph of CPU time
- Ox — interactive flame graph
- node --prof — built-in V8 profiler
- Chrome DevTools (node --inspect)

Common bottlenecks and fixes:

- Blocking synchronous operations → make async
- Unoptimized database queries → add indexes
- No caching → add Redis cache
- Expensive repeated computations → memoize
- Large JSON bodies → limit payload size
- Many event listeners → check for leaks

Key metrics to monitor:

- Event loop lag (> 50ms = problem)
- Memory usage (growing = leak)
- CPU usage (> 80% sustained = problem)
- Request latency percentiles (p50, p95, p99)

```
1 // 1. Profile first – find the bottleneck
2 // $ clinic doctor -- node app.js
3 // $ clinic flame -- node app.js
4
5 // 2. Enable compression
6 const compression = require('compression')
7 app.use(compression()) // gzip responses – 70-80% size reduction!
8
9 // 3. Cache expensive operations
10 const cache = new Map()
11 async function getProductsOptimized(category) {
12   const key = `products:${category}`
13   if (cache.has(key)) return cache.get(key)
14   const data = await db.products.find({ category })
15   cache.set(key, data)
16   setTimeout(() => cache.delete(key), 5 * 60 * 1000) // 5 min TTL
17   return data
18 }
19
20 // 4. Stream large responses instead of buffering
21 app.get('/export', async (req, res) => {
22   res.setHeader('Content-Type', 'text/csv')
```

```
23   const cursor = db.orders.find({}).cursor() // stream from DB
```

```

24   for await (const order of cursor) {
25       res.write(toCsvLine(order))
26   }
27   res.end()
28 })
29
30 // 5. Avoid synchronous operations in hot paths
31 // ■ Slow – reading file on EVERY request
32 app.get('/template', (req, res) => {
33     const html = fs.readFileSync('./template.html') // blocks event loop!
34     res.send(html)
35 })
36 // ■ Fast – read once at startup
37 const template = fs.readFileSync('./template.html', 'utf8') // startup only!
38 app.get('/template', (req, res) => res.send(template))
39
40 // 6. Set timeout for external calls
41 const axios = require('axios').create({ timeout: 5000 }) // 5s timeout
42
43 // 7. Parse JSON efficiently (large payloads)
44 app.use(express.json({ limit: '100kb' })) // reject payloads > 100KB

```

■ Interview Tip:

Run clinic doctor on your production-like load test before optimizing. It analyzes your app and tells you specifically what's wrong: event loop blocking, memory leak, unhandled rejection, etc. Don't guess — profile.

What is logging in Node.js? Best practices?

■ Answer

Proper logging is essential for debugging, monitoring, and auditing production systems.

Logging libraries:

- pino — fastest Node.js logger (structured JSON)
- winston — flexible, many transports
- bunyan — JSON logger with serializers
- morgan — HTTP request logging middleware

Log levels (in order of severity):

- trace — very detailed, for development
- debug — debugging information
- info — normal operations
- warn — potential issues, recoverable
- error — errors, unexpected but handled
- fatal — unrecoverable errors, process should exit

Structured logging best practices:

- JSON format — machine-readable, queryable
- Include: timestamp, level, message, requestId, userId, correlationId
- Never log: passwords, tokens, credit cards, SSNs
- Use log aggregation: ELK stack, Datadog, CloudWatch

```
1  const pino = require('pino')
2
3  // Configure pino logger
4  const logger = pino({
5    level: process.env.LOG_LEVEL || 'info',
6    transport: process.env.NODE_ENV === 'development'
7      ? { target: 'pino-pretty', options: { colorize: true } } // pretty in dev
8      : undefined, // JSON in production (for log aggregators)
9    serializers: {
10      req(req) {
11        return { method: req.method, url: req.url, remoteAddress: req.ip }
12      },
13      res(res) { return { statusCode: res.statusCode } },
14      err: pino.stdSerializers.err // properly serialize errors
15    },
16    redact: { // REMOVE sensitive fields
17      paths: ['req.headers.authorization', 'body.password', 'body.creditCard'],
18      censor: '[REDACTED]'
19    }
20  })
21
22  // Request logging middleware
23  app.use((req, res, next) => {
24    const start = Date.now()
```

```
25  const childLogger = logger.child({
```

```

26     requestId: req.id,
27     userId: req.user?.id
28   })
29   req.log = childLogger // attach to request
30
31   res.on('finish', () => {
32     const level = res.statusCode >= 500 ? 'error' : res.statusCode >= 400 ? 'warn' : 'info'
33     childLogger[level]({ req, res, duration: Date.now() - start }, 'request completed')
34   })
35   next()
36 })
37
38 // Usage in route handlers
39 app.get('/orders/:id', async (req, res) => {
40   req.log.info({ orderId: req.params.id }, 'fetching order')
41   const order = await Order.findById(req.params.id)
42   if (!order) {
43     req.log.warn({ orderId: req.params.id }, 'order not found')
44     return res.status(404).json({ error: 'Order not found' })
45   }
46   req.log.info({ orderId: order._id, status: order.status }, 'order fetched')
47   res.json(order)
48 })

```

■ Interview Tip:

Always use a structured logger (pino/winston) instead of `console.log()` in production. JSON logs are searchable in Kibana/CloudWatch. `console.log()` doesn't have log levels, rate limiting, or redaction. Pino is ~5x faster than winston.

What is PM2? How to use it in production?

■ Answer

PM2 is a production process manager for Node.js applications.

It handles: clustering, auto-restart, monitoring, log management, and zero-downtime deploys.

Core PM2 features:

- Keep alive — auto-restart on crash or high memory
- Cluster mode — use all CPU cores automatically
- Zero-downtime reload — rolling restart
- Log management — log files with rotation
- Monitoring — CPU and memory per worker
- Startup scripts — survive reboots
- Environment management — different configs per env

ecosystem.config.js:

- Define app configuration declaratively
- Multiple apps in one file
- Environment-specific settings

```
1 // ecosystem.config.js - PM2 configuration
2 module.exports = {
3   apps: [
4     {
5       name: 'my-api',
6       script: './dist/server.js',
7       instances: 'max', // use all CPUs
8       exec_mode: 'cluster', // cluster mode
9       watch: false, // don't watch in production!
10      max_memory_restart: '500M', // restart if > 500MB
11      restart_delay: 3000, // wait 3s before restart
12      exp_backoff_restart_delay: 100, // exponential backoff
13      env: { // development env
14        NODE_ENV: 'development',
15        PORT: 3000
16      },
17      env_production: { // production env
18        NODE_ENV: 'production',
19        PORT: 3000
20      },
21      log_date_format: 'YYYY-MM-DD HH:mm:ss Z',
22      error_file: './logs/err.log',
23      out_file: './logs/out.log',
24      merge_logs: true, // combine cluster worker logs
25      log_type: 'json'
26    }
27  ]
28 }
29
```

```
30 // Essential PM2 commands:
```

```
31 // pm2 start ecosystem.config.js --env production
32 // pm2 reload my-api          # zero-downtime rolling restart
33 // pm2 restart my-api         # hard restart (brief downtime)
34 // pm2 stop my-api
35 // pm2 delete my-api
36 // pm2 list                   # all processes
37 // pm2 monit                  # real-time CPU/memory
38 // pm2 logs my-api --lines 100 # view logs
39 // pm2 logs my-api --err       # only errors
40 // pm2 flush my-api            # clear logs
41 // pm2 startup                 # generate startup script
42 // pm2 save                    # save current processes
43
44 // Zero-downtime deploy script
45 // #!/bin/bash
46 // git pull origin main
47 // npm ci --production
48 // npm run build
49 // pm2 reload my-api ← rolling restart, no downtime!
```

■ Interview Tip:

pm2 reload is magic — it starts new workers before stopping old ones (rolling restart). Users never experience downtime during code deploys. Combined with pm2 startup && pm2 save, your Node.js app survives server reboots automatically.

What is libuv? Thread pool?

■ Answer

libuv is the C library that powers Node.js's asynchronous I/O and event loop. It abstracts OS-specific async I/O APIs (epoll, kqueue, IOCP) into a unified interface.

libuv components:

- Event Loop — 6-phase loop that processes async callbacks
- Thread Pool — fixed number of threads for blocking operations
- I/O Polling — OS-native async I/O (no threads needed!)
- Timers — setTimeout, setInterval implementation
- Process/Signal handling

Thread pool handles (NOT the event loop!):

- File system operations (fs module)
- DNS lookups (dns.lookup)
- Crypto operations (crypto.pbkdf2, etc.)
- User-defined C++ addons with thread pool usage

Thread pool does NOT handle:

- Network I/O (TCP, UDP, HTTP) — OS handles this natively

Configuring thread pool size:

- Default: 4 threads (UV_THREADPOOL_SIZE)
- Max: 1024 threads
- Set via environment variable before starting Node.js

```
1 // Demonstrating thread pool behavior
2 const crypto = require('crypto')
3 const os = require('os')
4
5 console.log('CPU cores:', os.cpus().length)
6 console.log('Thread pool size:', process.env.UV_THREADPOOL_SIZE || 4)
7
8 // These operations use the thread pool:
9 // Run 8 crypto operations simultaneously
10 const start = Date.now()
11 const tasks = Array.from({ length: 8 }, (_, i) => {
12   return new Promise((resolve) => {
13     crypto.pbkdf2('password', 'salt', 100000, 64, 'sha512', () => {
14       console.log(`Task ${i+1} done at ${Date.now() - start}ms`)
15       resolve()
16     })
17   })
18 })
19
20 // With default pool size 4:
21 // Tasks 1-4: start immediately (4 threads)
22 // Tasks 5-8: wait for a thread to free up
```

```
23 // You'll see 4 finish at ~400ms, then 4 more at ~800ms
```



```
24
25 // With UV_THREADPOOL_SIZE=8:
26 // All 8 start immediately
27 // All finish at ~400ms (2x faster!)
28
29 // Increase thread pool for crypto-heavy apps
30 // Before starting node:
31 // UV_THREADPOOL_SIZE=16 node app.js
32
33 // Or in Node.js (must be before first I/O):
34 process.env.UV_THREADPOOL_SIZE = Math.max(os.cpus().length, 8).toString()
35
36 // Network I/O does NOT use thread pool (OS handles it)
37 const http = require('http')
38 // 1000 concurrent HTTP requests = no thread pool needed
39 // They all use OS-level async I/O (epoll/kqueue/IOCP)
```

■ Interview Tip:

If you're building an auth service with bcrypt, increase UV_THREADPOOL_SIZE! Each bcrypt hash uses 1 thread. With 4 threads and 10 concurrent login requests, 6 requests queue behind the first 4. Set UV_THREADPOOL_SIZE=16 for crypto-heavy workloads.

What is N-API? Native addons?

■ Answer

N-API (Node-API) is a stable ABI interface for building native C/C++ addons for Node.js.

Why native addons?

- Performance-critical operations where JS is too slow
- Reuse existing C/C++ libraries (OpenCV, TensorFlow, etc.)
- Access OS-level APIs not exposed by Node.js
- Hardware interface (serial ports, GPIO)

N-API advantages over old nan/NAN:

- ABI stable — addon works across Node.js versions
- No recompilation needed when Node.js updates
- Official, maintained by Node.js core team

Popular native addons you already use:

- bcrypt (hashing — uses C++ for performance)
- sharp (image processing — uses libvips C library)
- node-canvas (Cairo graphics library)
- sqlite3 (SQLite C library)
- ioredis (C++ TCP implementation)

```
1 // You don't usually WRITE N-API addons — you USE them
2 // Understanding N-API explains why these libraries are fast
3
4 // bcrypt — uses C++ native addon for hashing
5 const bcrypt = require('bcrypt') // compiled C++ under the hood
6 const hash = await bcrypt.hash('password', 12) // runs in thread pool!
7
8 // sharp — image processing using libvips C library
9 const sharp = require('sharp') // wraps libvips via N-API
10 await sharp('./input.jpg')
11   .resize(400, 400)
12   .webp({ quality: 80 })
13   .toFile('./output.webp') // processes in thread pool
14
15 // node-addon-api: C++ addon with N-API (simplified)
16 // addon.cc:
17 // #include <napi.h>
18 // Napi::Value Add(const Napi::CallbackInfo& info) {
19 //   double a = info[0].As<Napi::Number>();
20 //   double b = info[1].As<Napi::Number>();
21 //   return Napi::Number::New(info.Env(), a + b);
22 // }
23 // Napi::Object Init(Napi::Env env, Napi::Object exports) {
24 //   exports.Set('add', Napi::Function::New(env, Add));
25 //   return exports;
26 // }
```

```
27 // NODE_API_MODULE(addon, Init)
```



```
28
29 // Using the addon in JS:
30 // const addon = require('./build/Release/addon')
31 // addon.add(2, 3) // 5 – runs native C++ code!
32
33 // Simpler: use node-addon-api package
34 // package.json: { 'gypfile': true }
35 // binding.gyp: { 'targets': [{ 'target_name': 'addon', 'sources': ['addon.cc'] }] }
36 // npm install → compiles C++ → node ./index.js
```

■ Interview Tip:

Understanding that libraries like sharp and bcrypt are native addons explains why they use the thread pool instead of the event loop — they're C++ code that runs in libuv's thread pool, not JavaScript in the event loop.

Q42 What is microservices architecture with Node.js?

■ Answer

Microservices split a large application into small, independent services. Each service handles one business domain and communicates via APIs or message queues.

When to use microservices:

- Large teams working on the same codebase independently
- Different services need different scaling
- Different parts need different tech stacks
- High availability requirements for specific features

Microservices communication:

- Synchronous: REST APIs, gRPC
- Asynchronous: Message queues (Kafka, RabbitMQ, BullMQ)

Common patterns:

- API Gateway — single entry point for all services
- Service Discovery — services find each other
- Circuit Breaker — prevent cascade failures
- Event Sourcing — services communicate via events

Microservices challenges:

- Distributed tracing — following a request across services
- Data consistency — no shared database
- Service discovery — where is each service?
- Network latency — in-process calls → HTTP calls

```
1 // Microservices with Node.js + Express
2
3 // API Gateway - routes to appropriate service
4 const httpProxy = require('http-proxy-middleware')
5
6 const services = {
7   users: process.env.USERS_SERVICE_URL || 'http://users:3001',
8   orders: process.env.ORDERS_SERVICE_URL || 'http://orders:3002',
9   products: process.env.PRODUCTS_SERVICE_URL || 'http://products:3003',
10  payments: process.env.PAYMENTS_SERVICE_URL || 'http://payments:3004'
11 }
12
13 // Route to appropriate microservice
14 app.use('/api/users', httpProxy.createProxyMiddleware({
15   target: services.users, changeOrigin: true,
16   pathRewrite: { '^/api/users': '' } // strip prefix
17 }))
18 app.use('/api/orders', httpProxy.createProxyMiddleware({ target: services.orders, changeOrigin: true,
19   pathRewrite: { '^/api/orders': '' } // strip prefix
20 }))
21 // Circuit Breaker pattern - prevent cascade failures
```

```
21 const CircuitBreaker = require('opossum')
```

```

22
23 async function callUserService(userId) {
24     return fetch(`${services.users}/users/${userId}`).then(r => r.json())
25 }
26
27 const breaker = new CircuitBreaker(callUserService, {
28     timeout: 3000,           // fail if takes > 3s
29     errorThresholdPercentage: 50, // open if 50% fail
30     resetTimeout: 30000      // try again after 30s
31 })
32
33 breaker.fallback((userId) => ({ id: userId, name: 'Unknown', cached: true }))
34 breaker.on('open', () => logger.warn('Circuit opened! User service unreachable'))
35 breaker.on('halfOpen', () => logger.info('Trying user service again...'))
36 breaker.on('close', () => logger.info('User service recovered!'))
37
38 // Service-to-service auth
39 const internalClient = axios.create({
40     headers: { 'X-Internal-Secret': process.env.INTERNAL_SECRET },
41     timeout: 5000
42 })

```

■ Interview Tip:

The Circuit Breaker pattern is essential for microservices. Without it, if the User Service is down, every request that needs user data queues up, threads/connections exhaust, and the entire system crashes. Circuit breaker fails fast and serves fallback data.

What is API rate limiting strategies?

■ Answer

Rate limiting protects APIs from abuse, DDoS attacks, and ensures fair usage.

Rate limiting strategies:

- Fixed Window — N requests per time window (simple, burst at window edge)
- Sliding Window Log — track timestamps of each request (accurate, memory-heavy)
- Sliding Window Counter — approximate sliding window (good balance)
- Token Bucket — tokens refill at rate R, consume 1 per request (allows bursts)
- Leaky Bucket — process at fixed rate (strictly limits output rate)

Where to rate limit:

- API Gateway/Nginx — before hitting Node.js
- Express middleware — per route, per user
- Database layer — connection pool limits

What to rate limit by:

- IP address — prevent abuse from single IP
- User ID — prevent API key abuse
- API key — per-client limits
- Route — stricter limits for expensive endpoints

```

1  const redis = require('ioredis').createClient()
2
3  // Sliding window rate limiter (most accurate)
4  async function rateLimitCheck(identifier, limit, windowMs) {
5      const now = Date.now()
6      const windowStart = now - windowMs
7      const key = `rl:${identifier}`
8
9      // Use Redis sorted set for sliding window
10     const pipe = redis.pipeline()
11     pipe.zremrangebyscore(key, 0, windowStart) // remove expired
12     pipe.zadd(key, now, `${now}-${Math.random()}`) // add this request
13     pipe.zcount(key, windowStart, '+inf') // count in window
14     pipe.pexpire(key, windowMs) // set TTL
15     const [[,],[,],[count]] = await pipe.exec()
16
17     return {
18         allowed: count <= limit,
19         count, limit,
20         remaining: Math.max(0, limit - count),
21         retryAfter: count > limit ? Math.ceil(windowMs / 1000) : 0
22     }
23 }
24
25 // Express middleware using above
26 const rateLimit = ({ limit, windowMs, keyFn, errorMessage }) => {

```

```

27 return async (req, res, next) => {

```

```

28  const key = keyFn ? keyFn(req) : req.ip
29  const { allowed, remaining, limit: l, retryAfter } = await rateLimitCheck(key, limit, windowMs)
30
31  res.setHeader('X-RateLimit-Limit', l)
32  res.setHeader('X-RateLimit-Remaining', remaining)
33  if (retryAfter) res.setHeader('Retry-After', retryAfter)
34
35  if (!allowed) {
36    return res.status(429).json({
37      error: errorMessage || 'Too many requests',
38      retryAfter
39    })
40  }
41  next()
42 }
43 }
44
45 // Apply different limits to different endpoints
46 app.post('/auth/login', rateLimit({
47   limit: 5, windowMs: 15 * 60 * 1000, // 5 attempts per 15min
48   keyFn: req => `login:${req.ip}`,
49   errorMessage: 'Too many login attempts. Wait 15 minutes.'
50 })))
51
52 app.post('/api/ai/complete', authenticate, rateLimit({
53   limit: 20, windowMs: 60 * 1000, // 20 AI requests per minute
54   keyFn: req => `ai:${req.user.id}` // per-user limit
55 })))

```

■ Interview Tip:

Sliding window is superior to fixed window because it prevents the 'burst at boundary' attack. With fixed window, a user can send N requests at 11:59:59 and N requests at 12:00:01 — 2N requests in 2 seconds. Sliding window prevents this.

What is WebSocket in Node.js?

■ Answer

WebSocket enables full-duplex (bidirectional) communication between client and server. Unlike HTTP, WebSocket keeps a persistent connection open for real-time data flow.

HTTP vs WebSocket:

- HTTP: client requests → server responds → connection closes
- WebSocket: persistent connection → either side can send anytime

Use cases:

- Real-time chat
- Live notifications
- Real-time dashboards (stock prices, metrics)
- Multiplayer games
- Collaborative editing (Google Docs style)

WebSocket vs Server-Sent Events (SSE):

- WebSocket: bidirectional, custom protocol, better for interactive
- SSE: server → client only, built on HTTP, simpler, auto-reconnect

Libraries:

- ws — lightweight, low-level
- Socket.io — ws + fallbacks + rooms/namespaces

```
1  const http = require('http')
2  const { WebSocketServer } = require('ws')
3  const express = require('express')
4
5  const app = express()
6  const server = http.createServer(app)
7  const wss = new WebSocketServer({ server })
8
9  // Track clients and their metadata
10 const clients = new Map() // userId → { ws, rooms, lastSeen }
11
12 wss.on('connection', (ws, req) => {
13   const userId = getUserIdFromRequest(req) // parse from query or cookie
14   clients.set(userId, { ws, rooms: new Set(), lastSeen: Date.now() })
15
16   // Send initial state
17   ws.send(JSON.stringify({ type: 'CONNECTED', userId, timestamp: Date.now() }))
18
19   // Handle messages from client
20   ws.on('message', async (rawMessage) => {
21     try {
22       const message = JSON.parse(rawMessage.toString())
23
24       switch(message.type) {
```

```
25     case 'CHAT':
```



```

26     await handleChat(userId, message)
27     break
28   case 'JOIN_ROOM':
29     clients.get(userId).rooms.add(message.room)
30     broadcastToRoom(message.room, { type: 'USER_JOINED', userId })
31     break
32   case 'LEAVE_ROOM':
33     clients.get(userId).rooms.delete(message.room)
34     break
35   case 'PING':
36     ws.send(JSON.stringify({ type: 'PONG', timestamp: Date.now() }))
37     break
38   }
39 } catch(err) {
40   ws.send(JSON.stringify({ type: 'ERROR', message: err.message }))
41 }
42 })
43
44 ws.on('close', (code, reason) => {
45   clients.delete(userId)
46   broadcastOnline() // update online count for all
47 })
48
49 ws.on('error', (err) => console.error(`WebSocket error for ${userId}:`, err))
50 })
51
52 // Heartbeat – detect stale connections
53 setInterval(() => {
54   wss.clients.forEach(ws => {
55     if (ws.readyState === ws.OPEN) ws.ping() // check if alive
56   })
57 }, 30000)

```

■ Interview Tip:

WebSocket connections don't automatically reconnect. Client should implement reconnection with exponential backoff: try after 1s, 2s, 4s, 8s, etc. Socket.io handles this automatically — one of its main advantages over raw ws.

What is the HTTP/2 support in Node.js?

■ Answer

Node.js has built-in HTTP/2 support via the `http2` module.

HTTP/2 improves performance with multiplexing, header compression, and server push.

HTTP/2 advantages over HTTP/1.1:

- Multiplexing — multiple requests over ONE TCP connection (no head-of-line blocking)
- Header compression (HPACK) — smaller headers, faster
- Server push — proactively send resources to client
- Binary protocol — faster parsing than text-based HTTP/1.1
- Stream prioritization

HTTP/2 in practice:

- Requires HTTPS (SSL/TLS) in production
- Works with `http2` module or via Nginx (recommended for Node.js)
- Express supports HTTP/2 via `spdy` or native `http2` module

HTTP/2 + Nginx approach:

- Let Nginx handle HTTP/2 (it's better at it)
- Nginx terminates TLS and HTTP/2
- Forwards to Node.js over HTTP/1.1 (simpler)

```
1  const http2 = require('http2')
2  const fs = require('fs')
3
4  // HTTP/2 server (requires SSL/TLS!)
5  const server = http2.createSecureServer({
6    key: fs.readFileSync('./ssl/server.key'),
7    cert: fs.readFileSync('./ssl/server.crt')
8  })
9
10 server.on('stream', (stream, headers) => {
11   const method = headers[':method']
12   const path = headers[':path']
13
14   if (method === 'GET' && path === '/') {
15     // Server Push — send CSS before browser asks!
16     stream.pushStream({ ':path': '/styles.css' }, (err, pushStream) => {
17       if (!err) {
18         pushStream.respond({ ':status': 200, 'content-type': 'text/css' })
19         fs.createReadStream('./public/styles.css').pipe(pushStream)
20       }
21     })
22
23     // Main response
24     stream.respond({ ':status': 200, 'content-type': 'text/html' })
25     fs.createReadStream('./public/index.html').pipe(stream)
26   }
27 })
```



```
28
29  server.listen(443)
30
31  // =====
32  // OR: Let Nginx handle HTTP/2 (recommended!)
33  // nginx.conf:
34  // server {
35  //     listen 443 ssl http2;    ← HTTP/2 enabled
36  //     ssl_certificate /path/to/cert.pem;
37  //     ssl_certificate_key /path/to/key.pem;
38  //
39  //     location / {
40  //         proxy_pass http://localhost:3000; ← forward to Node.js over HTTP/1.1
41  //     }
42  // }
43  // Node.js listens on port 3000, serves HTTP/1.1
44  // Users get HTTP/2 benefits via Nginx!
```

■ Interview Tip:

The Nginx approach is recommended for production. Nginx is battle-tested at HTTP/2 (and HTTP/3) handling. Node.js focuses on application logic while Nginx handles TLS termination, HTTP/2, load balancing, and static file serving.

What is gRPC with Node.js?

■ Answer

gRPC is a high-performance RPC (Remote Procedure Call) framework by Google. It uses Protocol Buffers (protobuf) for serialization — much faster than JSON.

gRPC vs REST:

- gRPC: binary (protobuf), 5-10x faster, strict schema, HTTP/2
- REST: text (JSON), widely supported, flexible, HTTP/1.1

When to use gRPC:

- Microservices communication (internal services)
- High-throughput scenarios (millions of RPC calls/second)
- Mobile clients (smaller payload)
- Streaming (server streaming, client streaming, bidirectional)

gRPC types:

- Unary — single request, single response
- Server streaming — single request, stream of responses
- Client streaming — stream of requests, single response
- Bidirectional — both client and server stream

```

1 // 1. Define service in .proto file
2 // user.proto:
3 // syntax = 'proto3';
4 // service UserService {
5 //   rpc GetUser(GetUserRequest) returns (User);
6 //   rpc ListUsers(ListUsersRequest) returns (stream User);
7 // }
8 // message User { string id = 1; string name = 2; string email = 3; }
9 // message GetUserRequest { string id = 1; }
10 // message ListUsersRequest { string role = 1; }
11
12 const grpc = require('@grpc/grpc-js')
13 const protoLoader = require('@grpc/proto-loader')
14
15 // Load proto definition
16 const packageDef = protoLoader.loadSync('./user.proto', {
17   keepCase: true, longs: String, enums: String
18 })
19 const userProto = grpc.loadPackageDefinition(packageDef).UserService
20
21 // gRPC Server
22 const server = new grpc.Server()
23 server.addService(userProto.service, {
24   async getUser({ request }, callback) {
25     try {
26       const user = await User.findById(request.id)
27       if (!user) return callback({ code: grpc.status.NOT_FOUND })

```

```

28     callback(null, { id: user._id.toString(), name: user.name, email: user.email })

```

```

29   } catch(err) {
30     callback({ code: grpc.status.INTERNAL, message: err.message })
31   }
32 },
33 listUsers({ request }, stream) { // server streaming!
34   const cursor = User.find({ role: request.role }).cursor()
35   cursor.on('data', user => stream.write({ id: user._id.toString(), name: user.name }))
36   cursor.on('end', () => stream.end())
37   cursor.on('error', err => stream.destroy(err))
38 }
39 })
40
41 server.bindAsync('0.0.0.0:50051', grpc.ServerCredentials.createInsecure(), () => {
42   server.start()
43   console.log('gRPC server on :50051')
44 })
45
46 // gRPC Client (in another service)
47 const client = new userProto('user-service:50051', grpc.credentials.createInsecure())
48 const user = await util.promisify(client.getUser.bind(client))({ id: '123' })

```

■ Interview Tip:

gRPC is the standard for microservice-to-microservice communication at companies like Netflix, Google, and Uber. The strict schema (protobuf) acts as a contract between services — breaking changes are caught immediately.

What is GraphQL with Node.js?

■ Answer

GraphQL is a query language for APIs that lets clients request exactly what data they need.

GraphQL vs REST:

- GraphQL: one endpoint, client specifies data shape, no over/under-fetching
- REST: multiple endpoints, server specifies data shape

GraphQL benefits:

- Over-fetching — REST returns all fields; GraphQL returns only requested
- Under-fetching — REST needs multiple requests; GraphQL gets all in one
- Self-documenting — schema is the documentation
- Real-time — subscriptions built-in

GraphQL drawbacks:

- Caching is harder (one endpoint, POST requests)
- N+1 query problem (use DataLoader to solve)
- Learning curve
- Overkill for simple APIs

```
1  const { ApolloServer, gql } = require('@apollo/server')
2  const { startStandaloneServer } = require('@apollo/server/standalone')
3
4  // Define schema
5  const typeDefs = gql`
6    type User {
7      id: ID!
8      name: String!
9      email: String!
10     posts: [Post!]!
11   }
12   type Post {
13     id: ID!
14     title: String!
15     content: String!
16     author: User!
17     likeCount: Int!
18   }
19   type Query {
20     user(id: ID!): User
21     users: [User!]!
22     post(id: ID!): Post
23   }
24   type Mutation {
25     createPost(title: String!, content: String!): Post!
26     likePost(postId: ID!): Post!
27   }
28 `
```

```
29
```

```

30 // Resolvers
31 const resolvers = {
32   Query: {
33     user: (_, { id }) => User.findById(id),
34     users: () => User.find({ active: true }),
35     post: (_, { id }) => Post.findById(id)
36   },
37   Mutation: {
38     createPost: async (_, { title, content }, { userId }) => {
39       if (!userId) throw new Error('Unauthorized')
40       return Post.create({ title, content, author: userId })
41     }
42   },
43   User: {
44     posts: (user) => Post.find({ author: user._id }) // N+1 problem!
45   }
46 }
47
48 // Fix N+1 with DataLoader
49 const DataLoader = require('dataloader')
50 const postsByAuthor = new DataLoader(async (authorIds) => {
51   const posts = await Post.find({ author: { $in: authorIds } })
52   return authorIds.map(id => posts.filter(p => p.author.toString() === id.toString()))
53 })
54
55 // Updated resolver using DataLoader (batches queries!)
56 // User: { posts: (user) => postsByAuthor.load(user._id) }
57
58 const server = new ApolloServer({ typeDefs, resolvers })
59 await startStandaloneServer(server, { listen: { port: 4000 } })

```

■ Interview Tip:

The N+1 problem in GraphQL: if you fetch 100 users, User.posts resolver runs 100 times = 100 DB queries. DataLoader batches these into ONE query: 'find all posts where author in [id1, id2, ..., id100]'. Always use DataLoader for nested resolvers.

What is Serverless Node.js? AWS Lambda?

■ Answer

Serverless means running code without managing servers — cloud provider handles infrastructure. AWS Lambda runs Node.js functions in response to events.

How Lambda works:

- You write a handler function
- Lambda invokes it when an event occurs
- Scales from 0 to thousands of instances automatically
- Pay only for execution time (not idle time)

Lambda event sources:

- API Gateway — HTTP requests
- S3 — file uploads
- DynamoDB Streams — database changes
- SQS/SNS — queue messages
- CloudWatch — scheduled events (cron)

Lambda limitations:

- Cold starts — 100-500ms latency on first invocation
- Max 15 minute execution time
- Max 10GB memory, 10GB /tmp storage
- Stateless — no persistent connections between invocations

```
1 // AWS Lambda handler
2 const mongoose = require('mongoose')
3
4 // Reuse DB connection between warm invocations
5 let dbConnection = null
6
7 async function connectDB() {
8   if (dbConnection && mongoose.connection.readyState === 1) {
9     return // reuse existing connection!
10  }
11  dbConnection = await mongoose.connect(process.env.MONGODB_URI, {
12    maxPoolSize: 5, // small pool for Lambda
13    serverSelectionTimeoutMS: 5000
14  })
15 }
16
17 // Lambda handler function
18 exports.handler = async (event, context) => {
19   context.callbackWaitsForEmptyEventLoop = false // don't wait for event loop!
20
21   try {
22     await connectDB() // reused on warm invocation!
23
24     // Handle different event sources
```

```
25 if (event.httpMethod) {
```



```

26 // API Gateway event
27 const { httpMethod, path, body, queryStringParameters } = event
28 const data = body ? JSON.parse(body) : null
29
30 if (httpMethod === 'GET' && path === '/users') {
31     const users = await User.find({ active: true }).lean()
32     return {
33         statusCode: 200,
34         headers: { 'Content-Type': 'application/json' },
35         body: JSON.stringify(users)
36     }
37 }
38 }
39
40 if (event.Records) {
41     // SQS/SNS event – process messages
42     for (const record of event.Records) {
43         const message = JSON.parse(record.body)
44         await processMessage(message)
45     }
46     return { statusCode: 200 }
47 }
48
49 } catch(err) {
50     return { statusCode: 500, body: JSON.stringify({ error: err.message }) }
51 }
52 }

```

■ Interview Tip:

Setting `context.callbackWaitsForEmptyEventLoop = false` is crucial for MongoDB/database connections in Lambda. Without it, Lambda waits for all async operations to complete before returning — but mongoose keeps connections open, causing timeouts.

Q49 What is the net.createServer for Unix sockets?

■ Answer

Unix domain sockets (UDS) are IPC mechanisms that are faster than TCP/IP for local communication. They avoid the TCP/IP stack overhead when communicating between processes on the same machine.

Unix sockets vs TCP (localhost):

- Unix sockets: 2-3x faster, no TCP/IP overhead, file-based
- TCP localhost: slower, uses network stack, port-based
- Unix sockets: only work on same machine

Use cases:

- PostgreSQL, Redis, Nginx — all support Unix sockets
- Nginx → Node.js: use Unix socket for proxy
- Node.js → PostgreSQL: use Unix socket for DB queries
- IPC between Node.js processes

File path as socket:

- Socket is a special file in the filesystem
- Must clean up (delete) socket file on exit

```
1  const net = require('net')
2  const fs = require('fs')
3
4  const SOCKET_PATH = '/tmp/node-app.sock'
5
6  // Clean up old socket file
7  try { fs.unlinkSync(SOCKET_PATH) } catch {}
8
9  // Create Unix domain socket server
10 const server = net.createServer((socket) => {
11   let buffer = ''
12
13   socket.on('data', (data) => {
14     buffer += data.toString()
15     // Handle newline-delimited JSON protocol
16     const lines = buffer.split('\n')
17     buffer = lines.pop() // incomplete last line
18
19     for (const line of lines) {
20       if (!line.trim()) continue
21       try {
22         const msg = JSON.parse(line)
23         handleMessage(socket, msg)
24       } catch (err) {
25         socket.write(JSON.stringify({ error: 'Invalid JSON' }) + '\n')
26       }
27     }
28   })
29 }
```

```

30 socket.on('end', () => console.log('Client disconnected'))
31 socket.on('error', (err) => console.error('Socket error:', err.message))
32 })
33
34 server.listen(SOCKET_PATH, () => {
35   fs.chmodSync(SOCKET_PATH, 0o777) // set permissions
36   console.log('Unix socket server listening at', SOCKET_PATH)
37 })
38
39 // Nginx proxy via Unix socket (nginx.conf):
40 // server {
41 //   listen 80;
42 //   location / {
43 //     proxy_pass http://unix:/tmp/node-app.sock;
44 //   }
45 // }
46
47 // PostgreSQL via Unix socket:
48 const { Pool } = require('pg')
49 const pgPool = new Pool({
50   host: '/var/run/postgresql', // Unix socket directory
51   database: 'myapp',
52   user: 'postgres'
53   // No port needed for Unix socket!
54 })

```

■ Interview Tip:

When Nginx and Node.js are on the same server, configure Nginx to proxy to Node.js via Unix socket instead of TCP. This reduces latency for every single request. In high-traffic apps (1000+ req/sec), this optimization adds up significantly.

What is the vm module in Node.js?

■ Answer

The vm module allows running JavaScript in a separate V8 context (sandbox). Useful for running untrusted code, implementing script engines, or hot-reloading configs.

vm module use cases:

- Evaluating user-provided expressions safely
- Configuration files with JavaScript logic (like webpack.config.js)
- Implementing REPL-like functionality
- Template engines

vm is NOT a security sandbox!

- Code can still access native modules via require if available
- Can break out with careful exploitation
- For true sandboxing: use vm2, isolated-vm, or separate processes

vm.Script — precompile for reuse:

- Compile once, run multiple times
- Faster than re-parsing each time

```
1  const vm = require('vm')
2
3  // Run code in isolated context
4  const context = vm.createContext({
5    console, // give access to console
6    Math,    // give access to Math
7    result: null // to capture output
8  })
9
10 // Run script in context
11 const code = `result = Math.sqrt(16) + Math.PI`
12 vm.runInContext(code, context)
13 console.log(context.result) // 4 + 3.14159... = 7.14159
14
15 // vm.Script - compile once, run many times
16 const script = new vm.Script('x * x + y * y')
17 for (let i = 0; i < 1000; i++) {
18   const ctx = vm.createContext({ x: i, y: i + 1 })
19   const result = script.runInContext(ctx)
20   // result = i^2 + (i+1)^2
21 }
22
23 // Safe expression evaluator for configs
24 function evaluateExpression(expr, variables) {
25   const sandbox = { ...variables, Math, Date }
26   const ctx = vm.createContext(sandbox)
27   return vm.runInContext(expr, ctx, {
28     timeout: 100, // 100ms max execution
```

```
29   filename: 'expression'
```

```

30  })
31  }
32
33  evaluateExpression('price * 0.9', { price: 1000 }) // 900
34  evaluateExpression('new Date().getFullYear()', {}) // 2024
35
36  // Module-style loading (like require but from string)
37  function requireFromString(code, filename = 'virtual.js') {
38    const module = { exports: {} }
39    const wrapper = `(function(module, exports) { ${code} })`
40    const script = new vm.Script(wrapper, { filename })
41    const ctx = vm.createContext({ require, process })
42    script.runInContext(ctx).call(module.exports, module, module.exports)
43    return module.exports
44  }

```

■ Interview Tip:

vm.runInContext with a timeout is how you safely run user-provided expressions in apps like calculators, formula builders, or config engines. Always set a timeout — without it, infinite loops would hang your server forever.

What is Express.js over raw http — internal analysis?

■ Answer

Understanding how Express wraps Node's http module helps you write better Express code.

What Express adds to `http.IncomingMessage` (request):

- `req.params` — URL parameters from route patterns
- `req.query` — parsed query string (automatic)
- `req.body` — parsed request body (via middleware)
- `req.cookies` — parsed cookies (via cookie-parser)
- `req.ip` — client IP (handles X-Forwarded-For)
- `req.path`, `req.protocol`, `req.secure`

What Express adds to `http.ServerResponse` (response):

- `res.json(obj)` — sets Content-Type and stringifies
- `res.send(data)` — smart content-type detection
- `res.status(code)` — chainable status setting
- `res.redirect(url)` — sets Location header + 302
- `res.cookie(name, val, options)` — sets cookie header
- `res.render(view, data)` — render template engine

Express request lifecycle:

HTTP request → app.use stack → router → route handler → response

```
1 // Raw http request body parsing
2 // (What express.json() does internally)
3 const http = require('http')
4
5 function parseJsonBody(req) {
6   return new Promise((resolve, reject) => {
7     let body = ''
8     req.on('data', chunk => {
9       body += chunk.toString()
10      if (body.length > 1e6) { // 1MB limit
11        reject(new Error('Body too large'))
12        req.socket.destroy()
13      }
14    })
15    req.on('end', () => {
16      try { resolve(JSON.parse(body)) }
17      catch(err) { reject(new Error('Invalid JSON')) }
18    })
19    req.on('error', reject)
20  })
21 }
22
23 // What Express does with route params
24 // Route: /users/:id/posts/:postId
25 // URL: /users/123/posts/456
```

```
26 // Express extracts: { id: '123', postId: '456' }
```

```

27
28 // Express route matching (simplified internal)
29 function matchRoute(pattern, url) {
30   const paramNames = []
31   const regexStr = pattern.replace(/:([\w]+)/g, (_, name) => {
32     paramNames.push(name)
33     return '([\\w-]+)'
34   })
35   const match = url.match(new RegExp(`^${regexStr}$`))
36   if (!match) return null
37   return Object.fromEntries(paramNames.map((name, i) => [name, match[i+1]]))
38 }
39
40 // What res.json() does internally:
41 // res.setHeader('Content-Type', 'application/json')
42 // res.end(JSON.stringify(obj))
43
44 // What res.status() does:
45 // res.statusCode = code; return res; // chainable!
46
47 // Understanding this helps debug:
48 // If Express middleware doesn't call next() and doesn't call res.end()
49 // → request hangs forever (no http.ServerResponse.end() called)

```

■ Interview Tip:

Understanding that every Express response eventually calls `res.end()` (Node's `http.ServerResponse.end()`) helps you understand why: 1) not calling `next()` hangs requests, 2) calling `next()` after `res.json()` causes 'headers already sent' error.

What is the REPL in Node.js?

■ Answer

REPL stands for Read-Eval-Print Loop — an interactive programming environment. Type code, press Enter, see result immediately. Like browser DevTools console.

Accessing Node.js REPL:

- Run node in terminal (no arguments)
- .help — list REPL commands
- .exit or Ctrl+D — quit
- .save filename — save session to file
- .load filename — load file into REPL
- .editor — multi-line editor mode
- _ — last evaluated value

Custom REPL:

- Create programmatic REPL for your app
- Inject app context (models, services)
- Useful for: admin tasks, debugging prod, data exploration

REPL use cases:

- Quickly test code snippets
- Inspect objects and prototypes
- Run one-off database operations
- Debug/explore APIs

```
1  const repl = require('repl')
2  const mongoose = require('mongoose')
3
4  // Create custom REPL with app context
5  async function startREPL() {
6    await mongoose.connect(process.env.MONGODB_URI)
7    console.log('Connected to database!')
8
9    const r = repl.start({
10     prompt: 'myapp> ',
11     useColors: true,
12     ignoreUndefined: true // don't print 'undefined'
13   })
14
15   // Inject models into REPL context
16   r.context.User = require('./models/User')
17   r.context.Product = require('./models/Product')
18   r.context.Order = require('./models/Order')
19
20   // Inject helper functions
21   r.context.findUsers = (query) => User.find(query).lean()
22   r.context.countOrders = () => Order.countDocuments()
23
24   r.context.mongoose = mongoose
```



```

24
25 // Custom REPL commands
26 r.defineCommand('stats', {
27   help: 'Show database statistics',
28   async action() {
29     const [users, orders, products] = await Promise.all([
30       User.countDocuments(),
31       Order.countDocuments(),
32       Product.countDocuments()
33     ])
34     console.log(`Users: ${users}, Orders: ${orders}, Products: ${products}`)
35     this.displayPrompt()
36   }
37 })
38
39 r.on('exit', () => {
40   mongoose.disconnect()
41   process.exit(0)
42 })
43
44 console.log('REPL started! Available: User, Product, Order, findUsers, countOrders')
45 console.log('Commands: .stats .help .exit')
46 }
47
48 startREPL()
49
50 // Usage in terminal:
51 // > await User.findOne({ email: 'admin@test.com' })
52 // > await countOrders()
53 // > .stats
54 // > await Product.updateMany({}, { $inc: { price: 10 } })

```

■ Interview Tip:

A custom REPL is a powerful admin tool. Instead of writing one-off scripts, have a REPL connected to your production database (read-only!) where you can interactively explore data. Much safer than modifying production code for admin tasks.

What is the readline module?

■ Answer

The readline module provides an interface for reading input from a Readable stream line by line. Commonly used for CLI tools and reading large files line by line.

Use cases:

- CLI prompts — ask user for input
- Processing large files — line by line without loading all into memory
- Building interactive command-line tools
- Reading CSV files

readline vs streams:

- readline: convenient for line-by-line text processing
- streams: lower level, better for binary or when you need chunks

async iterator support:

- Node.js 11+ — readline interface is async iterable
- Use for await...of to iterate lines

```
1  const readline = require('readline')
2  const fs = require('fs')
3
4  // Read user input in CLI
5  const rl = readline.createInterface({
6    input: process.stdin,
7    output: process.stdout
8  })
9
10 function prompt(question) {
11   return new Promise(resolve => rl.question(question, resolve))
12 }
13
14 async function setupWizard() {
15   const name = await prompt('Enter your name: ')
16   const email = await prompt('Enter your email: ')
17   const role = await prompt('Enter role (admin/user): ')
18
19   console.log('\nCreating user...')
20   const user = await User.create({ name, email, role })
21   console.log('User created:', user._id)
22
23   rl.close()
24 }
25
26 // Process large file line by line (memory efficient!)
27 async function processLargeCSV(filename) {
28   const fileStream = fs.createReadStream(filename)
29   const rl = readline.createInterface({ input: fileStream, crlfDelay: Infinity })
```

```

31 let lineNum = 0
32 let headers = null
33 const batchSize = 100
34 let batch = []
35
36 // Async iterator (Node 11+)
37 for await (const line of rl) {
38   if (lineNum === 0) {
39     headers = line.split(',').map(h => h.trim())
40   } else {
41     const values = line.split(',')
42     const record = Object.fromEntries(headers.map((h, i) => [h, values[i]?.trim()]))
43     batch.push(record)
44
45     if (batch.length >= batchSize) {
46       await processOrInsertBatch(batch) // insert batch to DB
47       batch = []
48       console.log(`Processed ${lineNum} lines`)
49     }
50   }
51   lineNum++
52 }
53
54 if (batch.length > 0) await processOrInsertBatch(batch) // remaining
55 console.log(`Total processed: ${lineNum - 1} records`)
56 }
57
58 // Processes 1M+ row CSV with < 50MB memory!
59 processLargeCSV('./million-users.csv')

```

■ Interview Tip:

The async iterator approach for readline is elegant and memory-efficient. Processing a 10GB CSV file with for await...of and batched inserts can be done with < 50MB RAM. Without streaming, loading the file first would require 10GB of RAM.

What is the tls module? HTTPS in Node.js?

■ Answer

The `tls` module provides Transport Layer Security (TLS) and SSL. HTTPS is built on TLS — it encrypts HTTP traffic.

Creating HTTPS server:

- Require SSL certificate and private key
- Use `https.createServer()` or `tls.createServer()`
- For development: self-signed cert (mkcert tool)
- For production: Let's Encrypt (free) or paid CA

TLS configuration options:

- `cert` — certificate file content
- `key` — private key file content
- `ca` — Certificate Authority chain
- `minVersion` — minimum TLS version (TLSv1.2 minimum)
- `ciphers` — allowed cipher suites

Best practice: let Nginx handle TLS

- Nginx terminates TLS, forwards HTTP to Node.js
- Node.js doesn't need TLS at all
- Simpler, better performance, easier cert management

```
1  const https = require('https')
2  const fs = require('fs')
3  const express = require('express')
4
5  const app = express()
6  app.get('/', (req, res) => res.json({ secure: true }))
7
8  // Load certificates
9  const sslOptions = {
10     key: fs.readFileSync('./ssl/server.key'),
11     cert: fs.readFileSync('./ssl/server.crt'),
12     // ca: fs.readFileSync('./ssl/ca.crt'), // CA chain if needed
13     minVersion: 'TLSv1.2', // reject TLS 1.0 and 1.1
14     ciphers: [ // secure cipher suites only
15       'ECDHE-ECDSA-AES128-GCM-SHA256',
16       'ECDHE-RSA-AES128-GCM-SHA256',
17       'ECDHE-ECDSA-AES256-GCM-SHA384',
18     ].join(':'),
19   }
20
21  const httpsServer = https.createServer(sslOptions, app)
22  httpsServer.listen(443, () => console.log('HTTPS server on :443'))
23
24  // Redirect HTTP to HTTPS
25  const http = require('http')
```

```
26  http.createServer((req, res) => {
```

```

27   res.writeHead(301, { Location: `https://${req.headers.host}${req.url}` })
28   res.end()
29 }).listen(80)
30
31 // TLS client – verify server certificate
32 const tls = require('tls')
33 const tlsSocket = tls.connect({
34   host: 'api.example.com',
35   port: 443,
36   rejectUnauthorized: true // verify cert! (default: true)
37   // Set false ONLY in development – NEVER in production
38 }, () => {
39   console.log('TLS connected, authorized:', tlsSocket.authorized)
40   tlsSocket.write('GET / HTTP/1.1\r\nHost: api.example.com\r\n\r\n')
41 })
42
43 // Certificate transparency monitoring
44 app.use((req, res, next) => {
45   res.setHeader('Expect-CT', 'max-age=86400, enforce') // cert transparency
46   res.setHeader('Strict-Transport-Security', 'max-age=31536000; includeSubDomains')
47   next()
48 })

```

■ Interview Tip:

For production, NEVER set `rejectUnauthorized: false` when connecting to external HTTPS services — it disables certificate verification and makes your app vulnerable to man-in-the-middle attacks. The only valid use is development with self-signed certs.

What is Node.js in Kubernetes/Docker?

■ Answer

Containerizing Node.js apps for Kubernetes deployment requires specific considerations.

Node.js in Docker best practices:

- Multi-stage builds — smaller images
- Run as non-root user
- Use node:alpine images
- Copy package*.json first (layer caching)
- .dockerignore (node_modules, .env, .git)
- Health check endpoint

Node.js in Kubernetes:

- Implement /health and /ready endpoints
- Handle SIGTERM for graceful shutdown
- Set resource limits (CPU, memory)
- Use HPA for auto-scaling
- Use ConfigMap/Secret for environment variables

12-Factor App principles for Node.js:

- Store config in environment variables
- Treat backing services as attached resources
- Export services via port binding
- Scale out via process model

```
1 # Optimized Dockerfile for Node.js
2 FROM node:20-alpine AS deps
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci --only=production && npm cache clean --force
6
7 FROM node:20-alpine AS builder
8 WORKDIR /app
9 COPY package*.json ./
10 RUN npm ci
11 COPY . .
12 RUN npm run build
13
14 FROM node:20-alpine AS production
15 WORKDIR /app
16 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
17 USER appuser
18 ENV NODE_ENV=production PORT=3000
19 COPY --from=deps --chown=appuser /app/node_modules ./node_modules
20 COPY --from=builder --chown=appuser /app/dist ./dist
21 COPY package.json .
22 EXPOSE 3000
```

```
23 HEALTHCHECK --interval=30s --timeout=3s --start-period=30s \
```

```

24  CMD node -e 'require(http).get(http://localhost:3000/health, r => process.exit(r.statusCode ===
25  CMD ['node', 'dist/server.js']
26
27  # Kubernetes deployment
28  # containers:
29  # - name: api
30  #   image: myapp:v1.2.3
31  #   livenessProbe:
32  #     httpGet: { path: /health, port: 3000 }
33  #     initialDelaySeconds: 30
34  #   readinessProbe:
35  #     httpGet: { path: /ready, port: 3000 }
36  #     initialDelaySeconds: 10
37  #   resources:
38  #     requests: { memory: 256Mi, cpu: 100m }
39  #     limits: { memory: 512Mi, cpu: 500m }
40
41  // Node.js /ready endpoint – crucial for K8s
42  let isReady = false
43  mongoose.connect(MONGO_URI).then(() => { isReady = true })
44
45  app.get('/ready', (req, res) => {
46    if (!isReady) return res.status(503).json({ status: 'not ready', reason: 'db not connected' })
47    res.json({ status: 'ready' })
48  })

```

■ Interview Tip:

The readiness probe /ready endpoint is critical. Without it, Kubernetes routes traffic to a pod before the database is connected, causing errors. The /ready endpoint says 'I am ready to receive traffic' — only return 200 when all dependencies are up.

What is the Node.js security best practices summary?

■ Answer

Security in Node.js requires defense in depth — multiple layers of protection.

Input validation:

- Validate ALL input (body, params, query) with Zod/Joi
- Never trust user input
- Sanitize HTML to prevent XSS (DOMPurify)
- Parameterized queries to prevent SQL injection
- express-mongo-sanitize for NoSQL injection

Authentication & authorization:

- bcrypt for passwords (never MD5/SHA1)
- JWT with short expiry + refresh tokens
- HttpOnly cookies to prevent XSS token theft
- SameSite=Strict cookies to prevent CSRF
- Rate limit auth endpoints

Dependencies:

- npm audit regularly
- Update dependencies frequently
- Use dependabot/renovate for automated updates
- Avoid packages with known vulnerabilities

Infrastructure:

- Run as non-root in Docker
- helmet.js security headers
- HTTPS everywhere
- Keep Node.js version up to date

```
1  const helmet = require('helmet')
2  const mongoSanitize = require('express-mongo-sanitize')
3  const hpp = require('hpp') // HTTP Parameter Pollution
4  const rateLimit = require('express-rate-limit')
5
6  // 1. Security headers
7  app.use(helmet())
8
9  // 2. CORS
10 app.use(cors({ origin: ['https://myapp.com'], credentials: true }))
11
12 // 3. Rate limiting
13 app.use('/api/', rateLimit({ windowMs: 15*60*1000, max: 100 }))
14
15 // 4. Body size limits
16 app.use(express.json({ limit: '100kb' })) // reject large payloads
17
```

```
18 // 5. NoSQL injection prevention
```



```

19 app.use(mongoSanitize()) // removes $ and . from query/body
20
21 // 6. HTTP parameter pollution
22 app.use(hpp()) // prevent ?param=1&param=2 attacks
23
24 // 7. Validate all input
25 // Use Zod schemas on every route
26
27 // 8. Never expose stack traces in production
28 app.use((err, req, res, next) => {
29   const isDev = process.env.NODE_ENV === 'development'
30   res.status(err.statusCode || 500).json({
31     error: err.isOperational ? err.message : 'Internal Server Error',
32     ...(isDev && { stack: err.stack })
33   })
34 })
35
36 // 9. Audit npm packages
37 // npm audit -- check for known vulnerabilities
38 // npm audit fix -- auto-fix where possible
39
40 // 10. Environment variables validation at startup
41 const { z } = require('zod')
42 const envSchema = z.object({
43   NODE_ENV: z.enum(['development', 'test', 'production']),
44   PORT: z.coerce.number().default(3000),
45   MONGODB_URI: z.string().url(),
46   JWT_SECRET: z.string().min(32),
47   REDIS_URL: z.string().url()
48 })
49 const env = envSchema.parse(process.env) // crash on startup if missing!

```

■ Interview Tip:

Validate environment variables at startup with Zod/Joi. If JWT_SECRET is missing, your app should fail to start — not silently proceed with no authentication. Crashing at startup with a clear error is better than running insecurely.

What is caching strategies in Node.js apps?

■ Answer

Caching stores copies of data in faster storage to reduce latency and database load.

Caching layers (fastest to slowest):

- In-process (JS object/LRU) — microseconds, single process
- Redis (remote memory) — ~0.5ms, shared across processes
- CDN — 10-100ms, geographic distribution
- Database query cache — varies

Cache patterns:

- Cache-aside (Lazy loading) — check cache first, miss → fetch → store
- Write-through — update both DB and cache on write
- Write-behind — update cache, async write to DB
- Read-through — cache sits in front of DB, fetches on miss

Cache invalidation (hardest problem!):

- TTL — expire after N seconds (simple but may serve stale data)
- Event-driven — invalidate specific keys on data changes
- Version key — include version in key (e.g., user:v2:123)

What NOT to cache:

- Sensitive data (credit cards, passwords)
- Highly dynamic data (stock prices, chat messages)
- Per-user personalized data (without user-scoped keys)

```

1  const redis = require('ioredis').createClient()
2  const LRU = require('lru-cache')
3
4  // L1: In-process LRU cache (microseconds)
5  const l1 = new LRU({
6    max: 500,
7    ttl: 1000 * 60, // 1 minute
8    maxSize: 10 * 1024 * 1024, // 10MB
9    sizeCalculation: v => Buffer.byteLength(JSON.stringify(v))
10 })
11
12 // Multi-level caching
13 class CacheService {
14   async get(key) {
15     // L1: instant
16     const l1val = l1.get(key)
17     if (l1val !== undefined) return { data: l1val, hit: 'L1' }
18
19     // L2: Redis
20     const l2val = await redis.get(key)
21     if (l2val) {
22       const parsed = JSON.parse(l2val)
23
24       l1.set(key, parsed) // populate L1

```

```

24     return { data: parsed, hit: 'L2' }
25   }
26
27   return { data: null, hit: null }
28 }
29
30 async set(key, value, ttlSeconds = 300) {
31   const serialized = JSON.stringify(value)
32   l1.set(key, value) // L1
33   await redis.setex(key, ttlSeconds, serialized) // L2
34 }
35
36 async invalidate(...keys) {
37   for (const key of keys) l1.delete(key) // clear L1
38   if (keys.length > 0) await redis.del(...keys) // clear L2
39 }
40
41 async invalidatePattern(pattern) {
42   const keys = await redis.keys(pattern)
43   if (keys.length > 0) {
44     keys.forEach(k => l1.delete(k))
45     await redis.del(...keys)
46   }
47 }
48 }
49
50 const cache = new CacheService()
51
52 // Usage in service
53 async function getProduct(id) {
54   const { data, hit } = await cache.get(`product:${id}`)
55   if (data) { metrics.cacheHit(hit); return data }
56   const product = await Product.findById(id).lean()
57   if (product) await cache.set(`product:${id}`, product, 3600)
58   return product
59 }
60
61 // Invalidate on update
62 async function updateProduct(id, changes) {
63   const product = await Product.findByIdAndUpdate(id, changes, { new: true })
64   await cache.invalidate(`product:${id}`) // specific key
65   await cache.invalidatePattern('products:list:*') // all list caches
66   return product
67 }

```

■ Interview Tip:

L1 (in-process) + L2 (Redis) multi-level caching is the pattern used by high-scale apps. L1 handles the hottest data at near-zero latency; L2 handles the broader dataset and survives server restarts. Track cache hit rates to measure effectiveness.

Node.js 130 Q — Complete! ■

■ All 57 Topic with detailed answers + code!

Next: Express.js (80 Q) → MongoDB (120 Q) → Auth & Security (60 Q)

■ SAVE — your Node.js interview bible

■■ REPOST — help your dev community

■ COMMENT 'JS950' — get all 950 Q PDF

■ FOLLOW Kaushal Singh — daily prep

Kaushal Singh

Full-Stack Dev | MERN + React Native | Kanmotech Pvt. Ltd.

1M+ Impressions | Node.js · Express · MongoDB · Interview Prep

Section 3/10 done | Next: Express 80 Q | Comment 'JS950'